JOSHUA_THOMAS_CAPSTONE_HEALTHCARE_PROJECT_final

March 10, 2026

[1]:
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib
from matplotlib import pyplot as plt
```

[2]:
```
print('Numpy Version',np.__version__)
print('Pandas Version',pd.__version__)
print('Seaborn Version',sns.__version__)
print('Matplotlib Version',matplotlib.__version__)
```

```
Numpy Version 1.24.3
Pandas Version 2.0.3
Seaborn Version 0.12.2
Matplotlib Version 3.7.2
```

[3]:
```
pip install folium
```

```
Requirement already satisfied: folium in c:\users\cifer\anaconda3\lib\site-
packages (0.17.0)
Requirement already satisfied: branca>=0.6.0 in
c:\users\cifer\anaconda3\lib\site-packages (from folium) (0.8.0)
Requirement already satisfied: jinja2>=2.9 in c:\users\cifer\anaconda3\lib\site-
packages (from folium) (3.1.2)
Requirement already satisfied: numpy in c:\users\cifer\anaconda3\lib\site-
packages (from folium) (1.24.3)
Requirement already satisfied: requests in c:\users\cifer\anaconda3\lib\site-
packages (from folium) (2.31.0)
Requirement already satisfied: xyzservices in c:\users\cifer\anaconda3\lib\site-
packages (from folium) (2022.9.0)
Requirement already satisfied: MarkupSafe>=2.0 in
c:\users\cifer\anaconda3\lib\site-packages (from jinja2>=2.9->folium) (2.1.1)
Requirement already satisfied: charset-normalizer<4,>=2 in
c:\users\cifer\anaconda3\lib\site-packages (from requests->folium) (2.0.4)
Requirement already satisfied: idna<4,>=2.5 in
c:\users\cifer\anaconda3\lib\site-packages (from requests->folium) (3.4)
Requirement already satisfied: urllib3<3,>=1.21.1 in
```

```
c:\users\cifer\anaconda3\lib\site-packages (from requests->folium) (1.26.16)
Requirement already satisfied: certifi>=2017.4.17 in
c:\users\cifer\anaconda3\lib\site-packages (from requests->folium) (2023.7.22)
Note: you may need to restart the kernel to use updated packages.


[notice] A new release of pip is available: 24.2 -> 24.3.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

[4]:
```
sns.set()
sns.set_palette(palette='deep')
import folium
from folium.plugins import FastMarkerCluster
```

[5]:
```
df= pd.read_csv(r'C:\Users\Cifer\OneDrive\Desktop\My Great Learning\CAPSTONE␣
↪PROJECT\Data.csv')

df.head()
```

[5]:
```
   applicant_id  years_of_insurance_with_us  regular_checkup_lasy_year  \
0          5000                           3                          1
1          5001                           0                          0
2          5002                           1                          0
3          5003                           7                          4
4          5004                           3                          1

   adventure_sports Occupation  visited_doctor_last_1_year cholesterol_level  \
0                 1    Salried                           2        125 to 150
1                 0    Student                           4        150 to 175
2                 0   Business                           4        200 to 225
3                 0   Business                           2        175 to 200
4                 0    Student                           2        150 to 175

   daily_avg_steps  age  heart_decs_history  …   smoking_status  \
0             4866   28                   1  …          Unknown
1             6411   50                   0  …  formerly smoked
2             4509   68                   0  …  formerly smoked
3             6214   51                   0  …          Unknown
4             4938   44                   0  …     never smoked

  Year_last_admitted   Location  weight covered_by_any_other_company  Alcohol  \
0                NaN    Chennai      67                            N     Rare
1                NaN     Jaipur      58                            N     Rare
2                NaN     Jaipur      73                            N    Daily
3                NaN    Chennai      71                            Y     Rare
4             2004.0  Bangalore      74                            N       No

   exercise  weight_change_in_last_one_year fat_percentage insurance_cost
```

```
0  Moderate                                 1              25           20978
1  Moderate                                 3              27            6170
2   Extreme                                 0              32           28382
3        No                                 3              37           27148
4   Extreme                                 0              34           29616

[5 rows x 24 columns]
```

[6]:
```
df = df.drop(columns=['applicant_id'])
```

[7]:
```
pd.options.display.max_columns = None
```

[8]:
```
df.head()
```

[8]:
```
   years_of_insurance_with_us  regular_checkup_lasy_year  adventure_sports  \
0                           3                          1                 1
1                           0                          0                 0
2                           1                          0                 0
3                           7                          4                 0
4                           3                          1                 0

  Occupation  visited_doctor_last_1_year cholesterol_level  daily_avg_steps  \
0    Salried                           2        125 to 150             4866
1    Student                           4        150 to 175             6411
2   Business                           4        200 to 225             4509
3   Business                           2        175 to 200             6214
4    Student                           2        150 to 175             4938

   age  heart_decs_history  other_major_decs_history  Gender  \
0   28                   1                         0    Male
1   50                   0                         0    Male
2   68                   0                         0  Female
3   51                   0                         0  Female
4   44                   0                         1    Male

   avg_glucose_level   bmi   smoking_status  Year_last_admitted   Location  \
0                 97  31.2          Unknown                 NaN    Chennai
1                212  34.2  formerly smoked                 NaN     Jaipur
2                166  40.4  formerly smoked                 NaN     Jaipur
3                109  22.9          Unknown                 NaN    Chennai
4                118  26.5     never smoked              2004.0  Bangalore

   weight covered_by_any_other_company Alcohol  exercise  \
0      67                            N    Rare  Moderate
1      58                            N    Rare  Moderate
2      73                            N   Daily   Extreme
3      71                            Y    Rare        No
```

```
4      74                              N      No   Extreme

   weight_change_in_last_one year  fat_percentage  insurance_cost
0                               1              25           20978
1                               3              27            6170
2                               0              32           28382
3                               3              37           27148
4                               0              34           29616
```

[9]:
```python
## Checking the shape of the data: Number of columns and rows

df.shape
```

[9]:
```
(25000, 23)
```

- The dataset contains 25,000 entries and 24 columns.

[10]:
```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25000 entries, 0 to 24999
Data columns (total 23 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   years_of_insurance_with_us      25000 non-null  int64
 1   regular_checkup_lasy_year       25000 non-null  int64
 2   adventure_sports                25000 non-null  int64
 3   Occupation                      25000 non-null  object
 4   visited_doctor_last_1_year      25000 non-null  int64
 5   cholesterol_level               25000 non-null  object
 6   daily_avg_steps                 25000 non-null  int64
 7   age                             25000 non-null  int64
 8   heart_decs_history              25000 non-null  int64
 9   other_major_decs_history        25000 non-null  int64
 10  Gender                          25000 non-null  object
 11  avg_glucose_level               25000 non-null  int64
 12  bmi                             24010 non-null  float64
 13  smoking_status                  25000 non-null  object
 14  Year_last_admitted              13119 non-null  float64
 15  Location                        25000 non-null  object
 16  weight                          25000 non-null  int64
 17  covered_by_any_other_company    25000 non-null  object
 18  Alcohol                         25000 non-null  object
 19  exercise                        25000 non-null  object
 20  weight_change_in_last_one year  25000 non-null  int64
 21  fat_percentage                  25000 non-null  int64
 22  insurance_cost                  25000 non-null  int64
dtypes: float64(2), int64(13), object(8)
```

```
memory usage: 4.4+ MB
```

- There are 14 columns of type int64, 2 columns of type float64, and 8 columns of type object.
- The dataset require cleaning and handling of missing values.

0.0.1 Finding missing values

[11]:
```
df.isnull().sum()
```

[11]:
```
years_of_insurance_with_us            0
regular_checkup_lasy_year             0
adventure_sports                      0
Occupation                            0
visited_doctor_last_1_year            0
cholesterol_level                     0
daily_avg_steps                       0
age                                   0
heart_decs_history                    0
other_major_decs_history              0
Gender                                0
avg_glucose_level                     0
bmi                                 990
smoking_status                        0
Year_last_admitted                11881
Location                              0
weight                                0
covered_by_any_other_company          0
Alcohol                               0
exercise                              0
weight_change_in_last_one_year        0
fat_percentage                        0
insurance_cost                        0
dtype: int64
```

- The column bmi has some missing values (24,010 non-null entries, so 990 values are missing).
- The column Year_last_admitted has significant missing data (13,119 non-null entries, meaning 11,881 values are missing).

[12]:
```
# 1. Fill missing 'bmi' values with the median
df['bmi'].fillna(df['bmi'].median(), inplace=True)

# 2. Fill missing 'Year_last_admitted' with 0 (assuming missing means never␣
↪admitted)
df['Year_last_admitted'].fillna(0, inplace=True)

# Verify that there are no missing values now
missing_values = df.isnull().sum()
```

```python
print("Missing values after treatment:\n", missing_values)
```

```
Missing values after treatment:
 years_of_insurance_with_us        0
regular_checkup_lasy_year         0
adventure_sports                  0
Occupation                        0
visited_doctor_last_1_year        0
cholesterol_level                 0
daily_avg_steps                   0
age                               0
heart_decs_history                0
other_major_decs_history          0
Gender                            0
avg_glucose_level                 0
bmi                               0
smoking_status                    0
Year_last_admitted                0
Location                          0
weight                            0
covered_by_any_other_company      0
Alcohol                           0
exercise                          0
weight_change_in_last_one_year    0
fat_percentage                    0
insurance_cost                    0
dtype: int64
```

- null values have been treated.
- For bmi, the median is used since it handles outliers better than the mean.
- For Year_last_admitted, 0 is used to represent missing entries.

[13]:
```python
df.insurance_cost.unique()
```

[13]:
```
array([20978,  6170, 28382, 27148, 29616, 39488, 37020, 22212,  8638,
       25914, 17276, 14808, 46892, 33318, 49360, 18510, 32084, 12340,
        3702,  9872, 55530,  4936, 51828, 23446,  7404, 57998, 38254,
       24680, 60466, 54296, 34552, 16042, 30850, 41956, 13574, 43190,
       48126, 19744, 56764, 35786, 11106, 61700, 44424, 40722, 59232,
       53062, 64168, 62934, 45658, 50594, 65402, 66636, 67870,  2468],
      dtype=int64)
```

[14]:
```python
df.describe(include="all")["insurance_cost"]
```

[14]:
```
count     25000.000000
unique             NaN
top                NaN
freq               NaN
```

```
mean      27147.407680
std       14323.691832
min        2468.000000
25%       16042.000000
50%       27148.000000
75%       37020.000000
max       67870.000000
Name: insurance_cost, dtype: float64
```

[15]:
```python
from sklearn.preprocessing import LabelEncoder

# 1. Label encode categorical columns
label_encoder = LabelEncoder()


# Columns that need to be label encoded (change based on actual column meanings)
categorical_columns = ['Gender', 'smoking_status', 'Location',␣
 ↪'covered_by_any_other_company',
                       'Alcohol', 'exercise']

for column in categorical_columns:
    df[column] = label_encoder.fit_transform(df[column])

# 2. Convert 'cholesterol_level' (or similar numeric columns stored as objects)␣
 ↪to numeric
# If there are non-numeric values that need to be handled, use errors='coerce'␣
 ↪to set invalid parsing as NaN
df['cholesterol_level'] = pd.to_numeric(df['cholesterol_level'],␣
 ↪errors='coerce')

# Print to verify the changes
print(df.dtypes)
```

```
years_of_insurance_with_us          int64
regular_checkup_lasy_year           int64
adventure_sports                    int64
Occupation                         object
visited_doctor_last_1_year          int64
cholesterol_level                 float64
daily_avg_steps                     int64
age                                 int64
heart_decs_history                  int64
other_major_decs_history            int64
Gender                              int32
avg_glucose_level                   int64
bmi                               float64
smoking_status                      int32
```

```
Year_last_admitted                 float64
Location                             int32
weight                               int64
covered_by_any_other_company         int32
Alcohol                              int32
exercise                             int32
weight_change_in_last_one_year       int64
fat_percentage                       int64
insurance_cost                       int64
dtype: object
```

[16]:
```
df.describe().T
```

[16]:

	count	mean	std	min	\
years_of_insurance_with_us	25000.0	4.089040	2.606612	0.0	
regular_checkup_lasy_year	25000.0	0.773680	1.199449	0.0	
adventure_sports	25000.0	0.081720	0.273943	0.0	
visited_doctor_last_1_year	25000.0	3.104200	1.141663	0.0	
cholesterol_level	0.0	NaN	NaN	NaN	
daily_avg_steps	25000.0	5215.889320	1053.179748	2034.0	
age	25000.0	44.918320	16.107492	16.0	
heart_decs_history	25000.0	0.054640	0.227281	0.0	
other_major_decs_history	25000.0	0.098160	0.297537	0.0	
Gender	25000.0	0.656880	0.474761	0.0	
avg_glucose_level	25000.0	167.530000	62.729712	57.0	
bmi	25000.0	31.357952	7.720963	12.3	
smoking_status	25000.0	1.377120	1.071777	0.0	
Year_last_admitted	25000.0	1051.562480	1000.751938	0.0	
Location	25000.0	6.932400	4.314647	0.0	
weight	25000.0	71.610480	9.325183	52.0	
covered_by_any_other_company	25000.0	0.303280	0.459684	0.0	
Alcohol	25000.0	1.441800	0.680582	0.0	
exercise	25000.0	0.994640	0.643792	0.0	
weight_change_in_last_one_year	25000.0	2.517960	1.690335	0.0	
fat_percentage	25000.0	28.812280	8.632382	11.0	
insurance_cost	25000.0	27147.407680	14323.691832	2468.0	

	25%	50%	75%	max
years_of_insurance_with_us	2.0	4.0	6.0	8.0
regular_checkup_lasy_year	0.0	0.0	1.0	5.0
adventure_sports	0.0	0.0	0.0	1.0
visited_doctor_last_1_year	2.0	3.0	4.0	12.0
cholesterol_level	NaN	NaN	NaN	NaN
daily_avg_steps	4543.0	5089.0	5730.0	11255.0
age	31.0	45.0	59.0	74.0
heart_decs_history	0.0	0.0	0.0	1.0
other_major_decs_history	0.0	0.0	0.0	1.0

```
Gender                              0.0      1.0      1.0      1.0
avg_glucose_level                 113.0    168.0    222.0    277.0
bmi                                26.3     30.5     35.3    100.6
smoking_status                      0.0      2.0      2.0      3.0
Year_last_admitted                  0.0   1992.0   2004.0   2018.0
Location                            3.0      7.0     11.0     14.0
weight                             64.0     72.0     78.0     96.0
covered_by_any_other_company        0.0      0.0      1.0      1.0
Alcohol                             1.0      2.0      2.0      2.0
exercise                            1.0      1.0      1.0      2.0
weight_change_in_last_one_year      1.0      3.0      4.0      6.0
fat_percentage                     21.0     31.0     36.0     42.0
insurance_cost                  16042.0  27148.0  37020.0  67870.0
```

[17]:
```
# Calculate the percentage of missing values for each column
missing_percentage = df.isnull().mean() * 100
print("Percentage of Missing Values:\n", missing_percentage)
```

```
Percentage of Missing Values:
 years_of_insurance_with_us          0.0
regular_checkup_lasy_year           0.0
adventure_sports                    0.0
Occupation                          0.0
visited_doctor_last_1_year          0.0
cholesterol_level                 100.0
daily_avg_steps                     0.0
age                                 0.0
heart_decs_history                  0.0
other_major_decs_history            0.0
Gender                              0.0
avg_glucose_level                   0.0
bmi                                 0.0
smoking_status                      0.0
Year_last_admitted                  0.0
Location                            0.0
weight                              0.0
covered_by_any_other_company        0.0
Alcohol                             0.0
exercise                            0.0
weight_change_in_last_one_year      0.0
fat_percentage                      0.0
insurance_cost                      0.0
dtype: float64
```

[18]:
```
# Calculate the percentage distribution for categorical columns
for col in df.select_dtypes(include='object').columns:
    print(f"Distribution for {col} (in %):\n", df[col].
 ↪value_counts(normalize=True) * 100)
```

```
    print("\n")
```

```
Distribution for Occupation (in %):
 Occupation
Student     40.676
Business    40.080
Salried     19.244
Name: proportion, dtype: float64
```

[19]:
```
# Display key percentiles for numerical columns
numerical_summary = df.describe(percentiles=[0.25, 0.5, 0.75])
print("Numerical Summary with Percentiles:\n", numerical_summary)
```

```
Numerical Summary with Percentiles:
        years_of_insurance_with_us  regular_checkup_lasy_year  \
count                25000.000000               25000.000000
mean                     4.089040                   0.773680
std                      2.606612                   1.199449
min                      0.000000                   0.000000
25%                      2.000000                   0.000000
50%                      4.000000                   0.000000
75%                      6.000000                   1.000000
max                      8.000000                   5.000000

       adventure_sports  visited_doctor_last_1_year  cholesterol_level  \
count      25000.000000                25000.000000                0.0
mean           0.081720                    3.104200                NaN
std            0.273943                    1.141663                NaN
min            0.000000                    0.000000                NaN
25%            0.000000                    2.000000                NaN
50%            0.000000                    3.000000                NaN
75%            0.000000                    4.000000                NaN
max            1.000000                   12.000000                NaN

       daily_avg_steps           age  heart_decs_history  \
count     25000.000000  25000.000000        25000.000000
mean       5215.889320     44.918320            0.054640
std        1053.179748     16.107492            0.227281
min        2034.000000     16.000000            0.000000
25%        4543.000000     31.000000            0.000000
50%        5089.000000     45.000000            0.000000
75%        5730.000000     59.000000            0.000000
max       11255.000000     74.000000            1.000000

       other_major_decs_history        Gender  avg_glucose_level  \
count              25000.000000  25000.000000       25000.000000
```

```
mean                  0.098160      0.656880         167.530000
std                   0.297537      0.474761          62.729712
min                   0.000000      0.000000          57.000000
25%                   0.000000      0.000000         113.000000
50%                   0.000000      1.000000         168.000000
75%                   0.000000      1.000000         222.000000
max                   1.000000      1.000000         277.000000

                bmi  smoking_status  Year_last_admitted      Location  \
count  25000.000000    25000.000000        25000.000000  25000.000000
mean      31.357952        1.377120         1051.562480      6.932400
std        7.720963        1.071777         1000.751938      4.314647
min       12.300000        0.000000            0.000000      0.000000
25%       26.300000        0.000000            0.000000      3.000000
50%       30.500000        2.000000         1992.000000      7.000000
75%       35.300000        2.000000         2004.000000     11.000000
max      100.600000        3.000000         2018.000000     14.000000

             weight  covered_by_any_other_company       Alcohol      exercise  \
count  25000.000000                   25000.000000  25000.000000  25000.000000
mean      71.610480                       0.303280      1.441800      0.994640
std        9.325183                       0.459684      0.680582      0.643792
min       52.000000                       0.000000      0.000000      0.000000
25%       64.000000                       0.000000      1.000000      1.000000
50%       72.000000                       0.000000      2.000000      1.000000
75%       78.000000                       1.000000      2.000000      1.000000
max       96.000000                       1.000000      2.000000      2.000000

       weight_change_in_last_one_year  fat_percentage  insurance_cost
count                     25000.000000    25000.000000    25000.000000
mean                          2.517960       28.812280    27147.407680
std                           1.690335        8.632382    14323.691832
min                           0.000000       11.000000     2468.000000
25%                           1.000000       21.000000    16042.000000
50%                           3.000000       31.000000    27148.000000
75%                           4.000000       36.000000    37020.000000
max                           6.000000       42.000000    67870.000000
```

[ ]:

```python

```

0.1 Exploratory Data Analysis

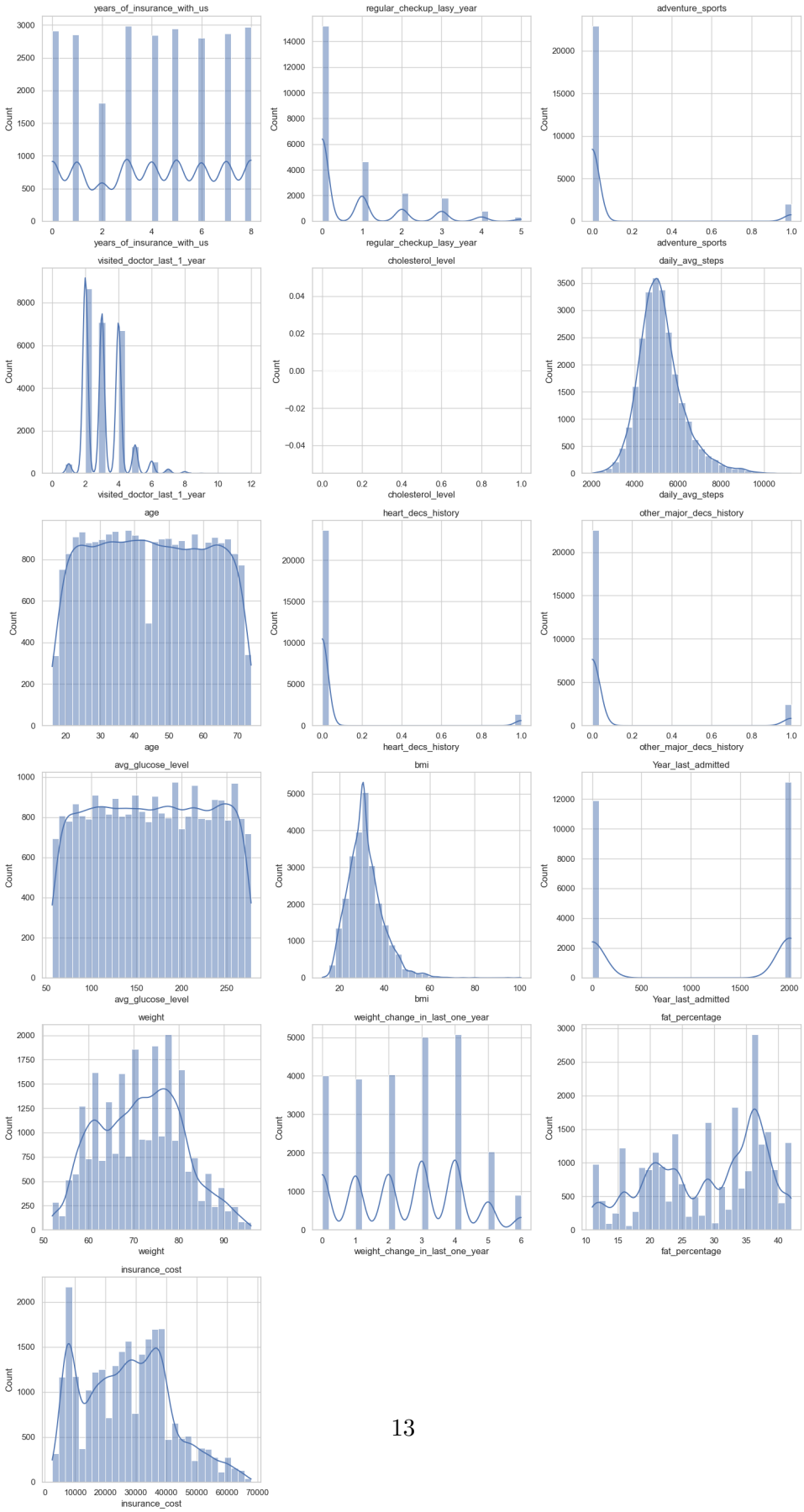
0.1.1 Univariate Analysis

[20]:

```python
# Set plot aesthetics
sns.set(style="whitegrid")

# Get list of numerical columns
```

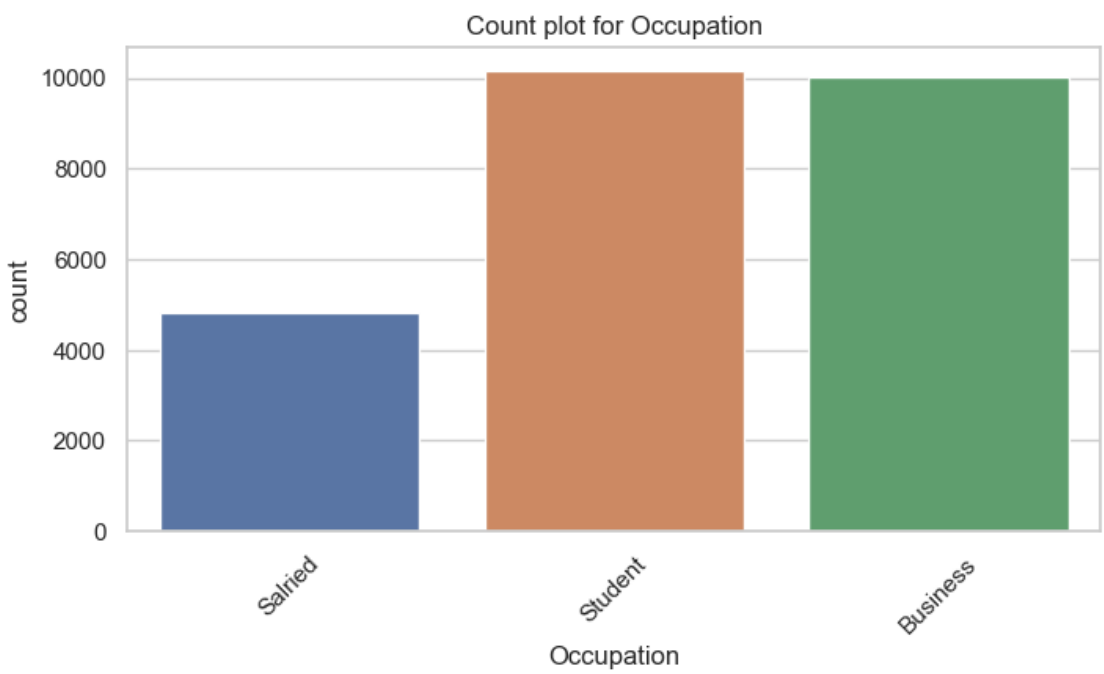
```python
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns

# Calculate the number of rows and columns for subplots
num_cols = 3  # Number of columns for the subplot grid
num_rows = (len(numerical_cols) // num_cols) + (len(numerical_cols) % num_cols␣
 ↪> 0)  # Add an extra row if needed

# Plot histograms for numerical columns
plt.figure(figsize=(15, 5 * num_rows))
plt.suptitle('Histograms for Numerical Variables', fontsize=20)

for i, col in enumerate(numerical_cols):
    plt.subplot(num_rows, num_cols, i + 1)  # Use dynamic rows and columns
    sns.histplot(df[col], kde=True, bins=30)
    plt.title(col)

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```

Histograms for Numerical Variables



- Many variables show right-skewed distributions, "daily_avg_steps", "insurance_cost", "weight", "bmi", and "fat_percentage". This suggests the possibility of outliers influencing statistical analysis. Right-skewed distributions often indicate the presence of potential outliers.
- Some variables have approximately normal distributions, Variables like "age" and "avg_glucose_level" appear to be closer to a normal distribution.
- Several variables exhibit highly skewed and/or multimodal distributions, Variables like "regular_checkup_lasy_year", "visited_doctor_last_1_year", "Location", and "weight_change_in_last_one_year" show distinct peaks or multiple modes, suggesting different underlying groups or patterns within the data.
- Some histograms, such as "Gender", "smoking_status", "covered_by_any_other_company", and "Alcohol", represent categorical variables as numerical data.

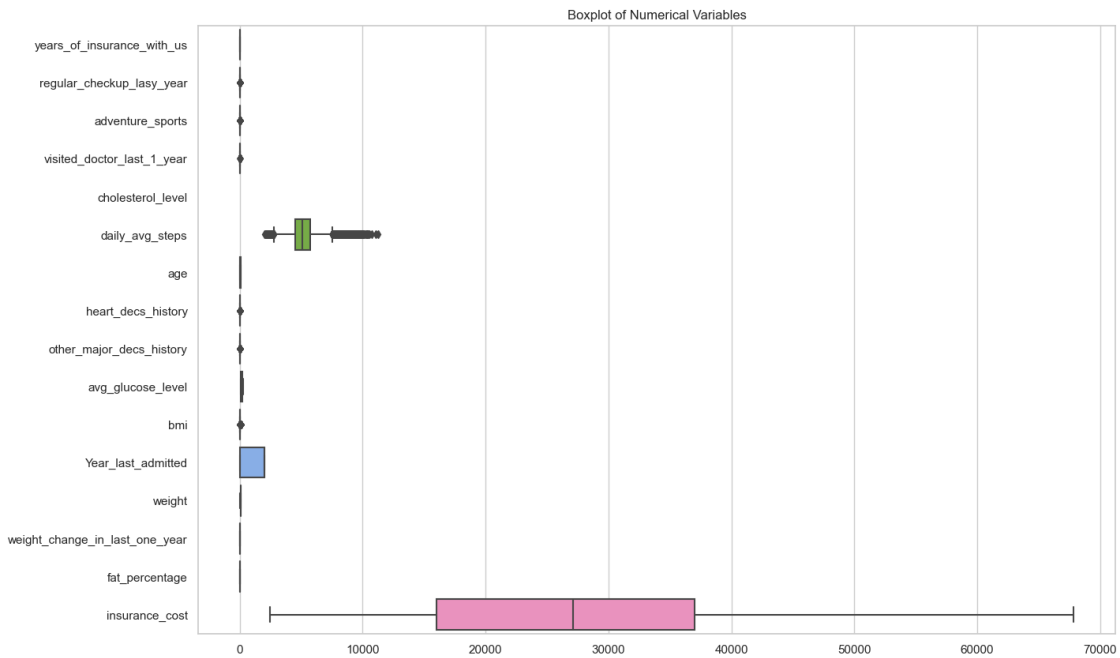
[22]:
```
# Identifying categorical columns in the dataset
categorical_cols = df.select_dtypes(include=['object']).columns.tolist()

# Plotting bar plots for each categorical column
for col in categorical_cols:
    plt.figure(figsize=(8, 4))
    sns.countplot(x=col, data=df)
    plt.title(f'Count plot for {col}')
    plt.xticks(rotation=45)
    plt.show()
```



- The graph illustrates the distribution of occupation across three categories: Salried, Student Occupation, and Business.
- Sallied: The blue bar indicates that 4,000 individuals are employed in the salaried sector.
- Student Occupation: The orange bar shows that 10,000 individuals are engaged in student occupations.
- Business: The green bar reveals that 10,000 individuals are involved in business.

0.1.2 Bivariate Analysis

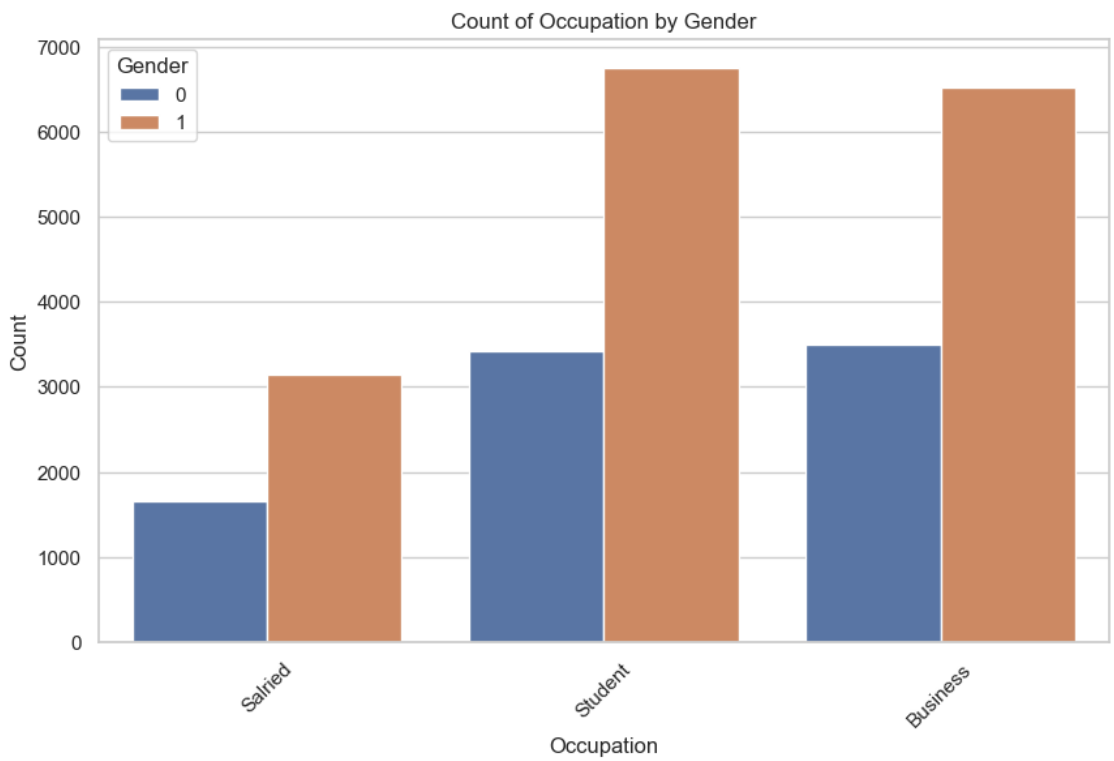
[24]:
```
plt.figure(figsize=(15, 10))
sns.boxplot(data=df[numerical_cols], orient="h")
plt.title('Boxplot of Numerical Variables')
plt.show()
```



- The boxplot for insurance_cost shows a significant right skew, indicating a long tail of high insurance costs.
- Several variables exhibit outliers. For example, "years_of_insurance_with_us", "regular_checkup_lasy_year", "visited_doctor_last_1_year", "daily_avg_steps", "age", and others all seem to have outliers.
- The boxplot highlights the distribution of various numerical variables, revealing skewness, outliers, and differences in ranges.

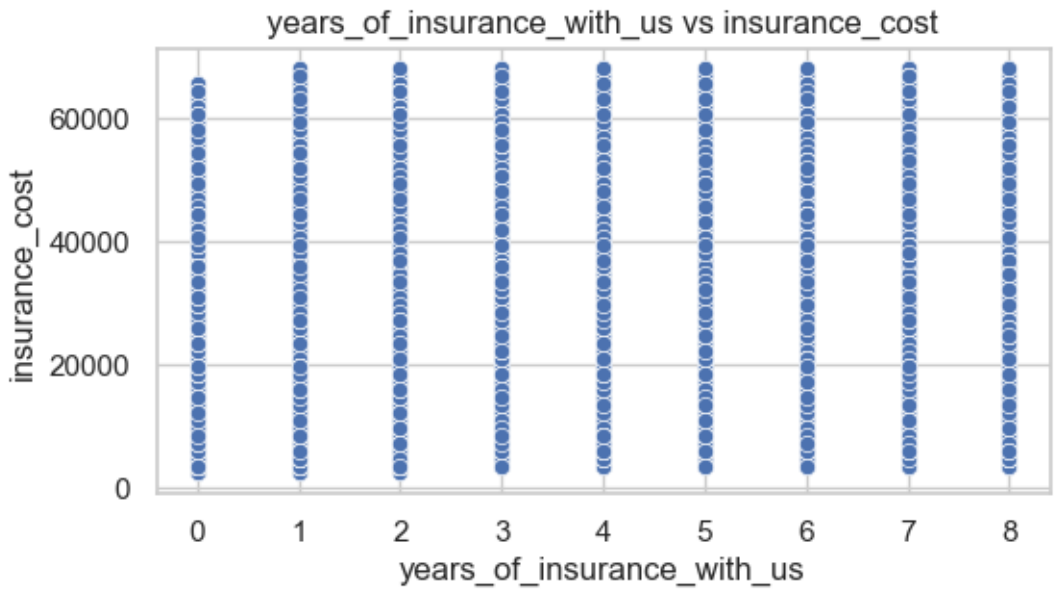
[25]:
```
# Bar plots for categorical variables
plt.figure(figsize=(10, 6))
sns.countplot(data=df, x='Occupation', hue='Gender')
plt.title('Count of Occupation by Gender')
```

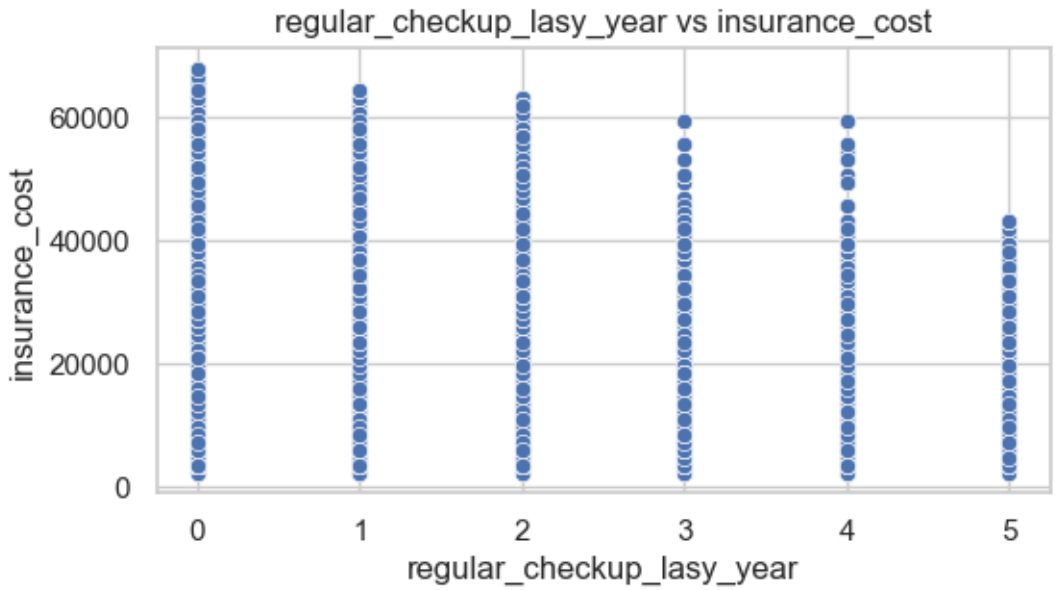
```
plt.xlabel('Occupation')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.legend(title='Gender')
plt.show()
```

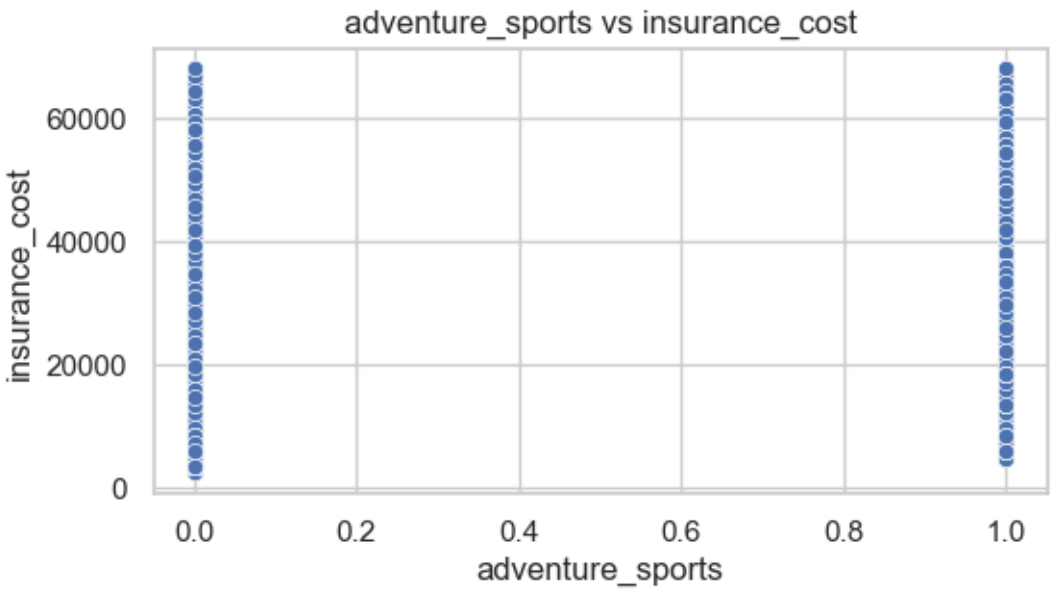


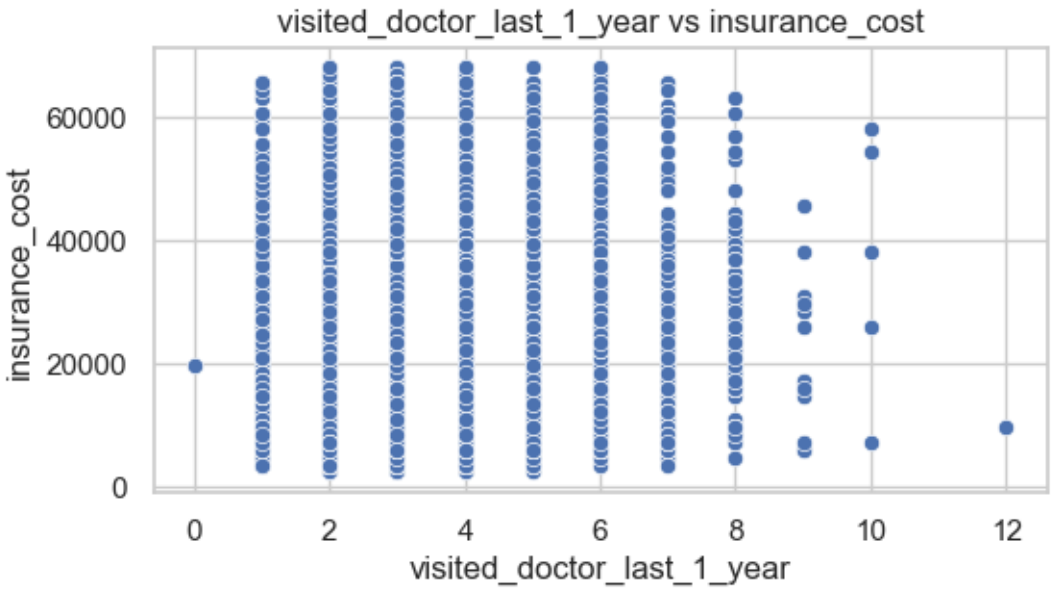
- The most immediate observation is that there are consistently more individuals identified as '1' (likely representing female) in each occupational category compared to those identified as '0' (likely male).
- The "Student" category has the most significant difference between genders, indicating a potential over-representation of females in that group.
- The proportion of males to females in both "Salaried" and "Business" categories appears relatively similar, although females still hold a larger share.

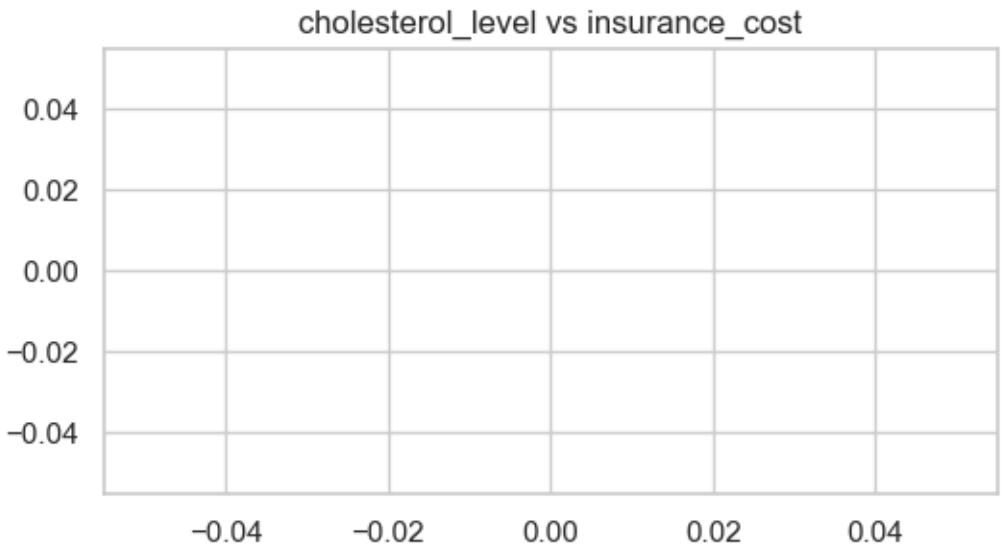
[26]:
```
for col in numerical_cols:
    plt.figure(figsize=(6,3))
    sns.scatterplot(x=col, y='insurance_cost', data=df)
    plt.title(f'{col} vs insurance_cost')
    plt.show()
```

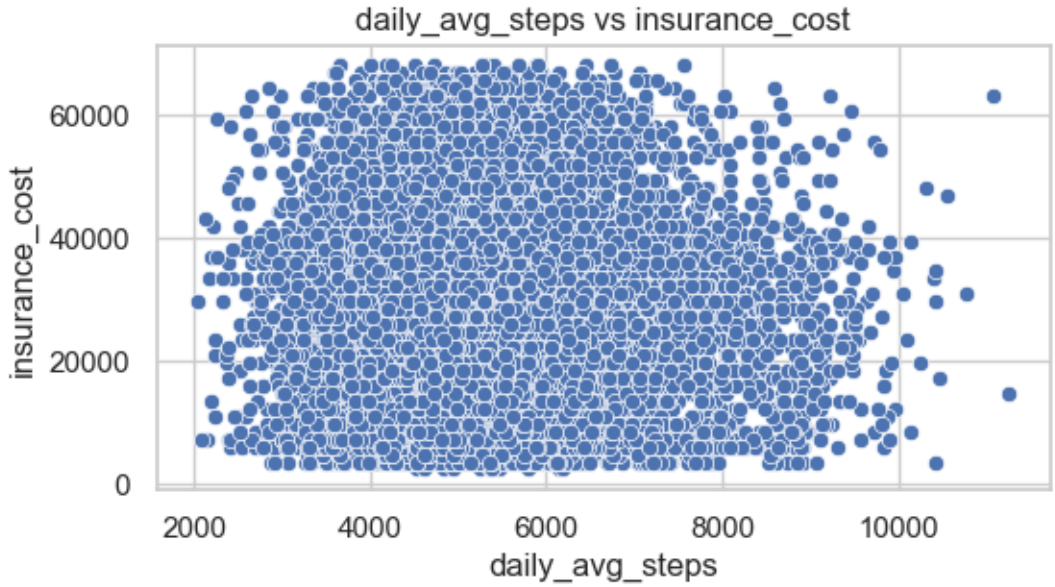

years_of_insurance_with_us vs insurance_cost
insurance_cost
0
20000
40000
60000
0
1
2
3
4
5
6
7
8
years_of_insurance_with_us

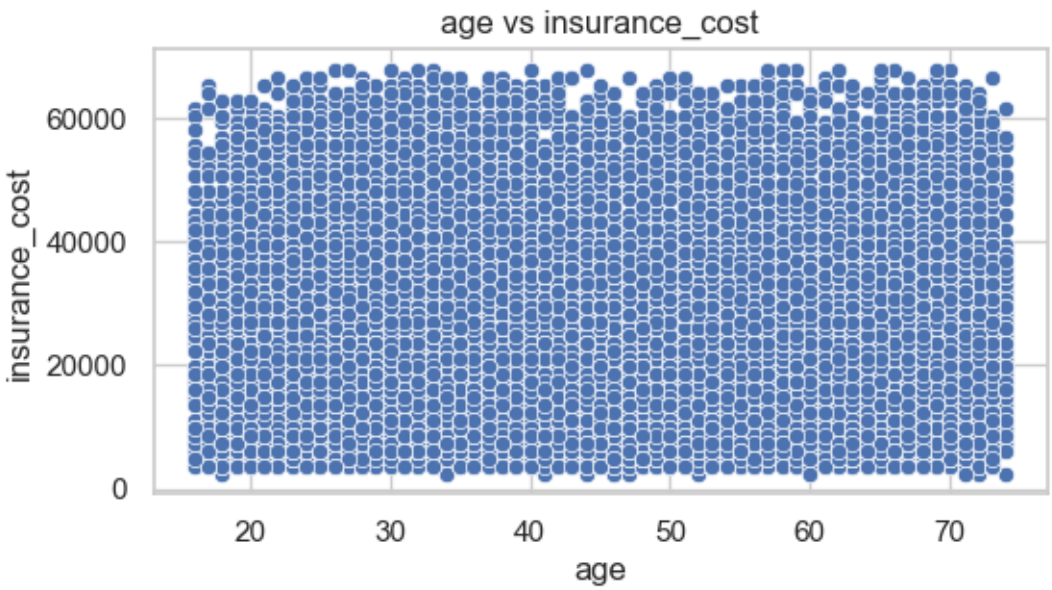

regular_checkup_lasy_year vs insurance_cost
insurance_cost
0
20000
40000
60000
0
1
2
3
4
5
regular_checkup_lasy_year

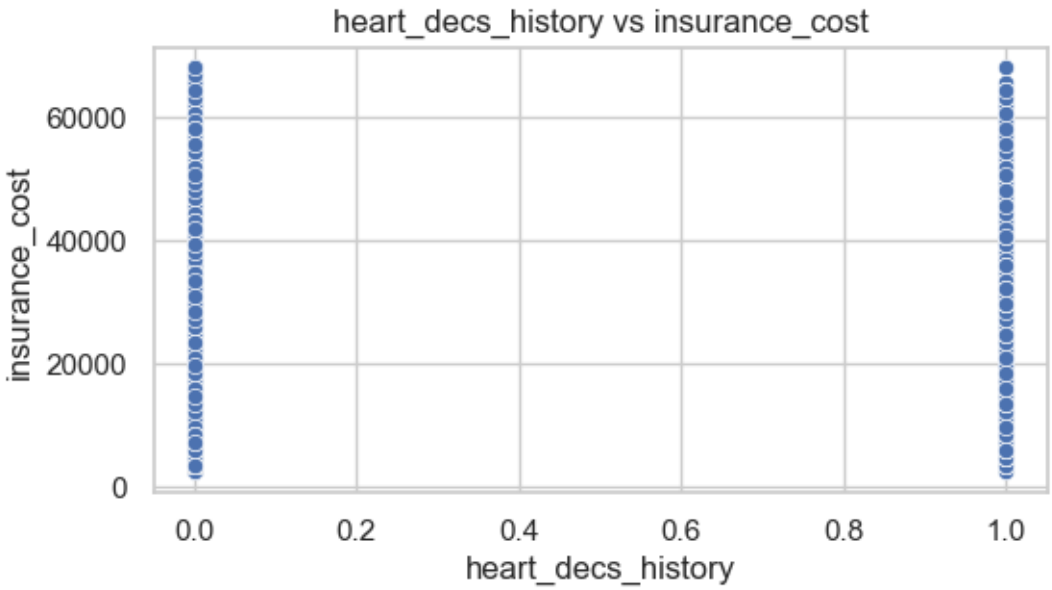

adventure_sports vs insurance_cost
insurance_cost
60000
40000
20000
0
0.0
0.2
0.4
0.6
0.8
1.0
adventure_sports

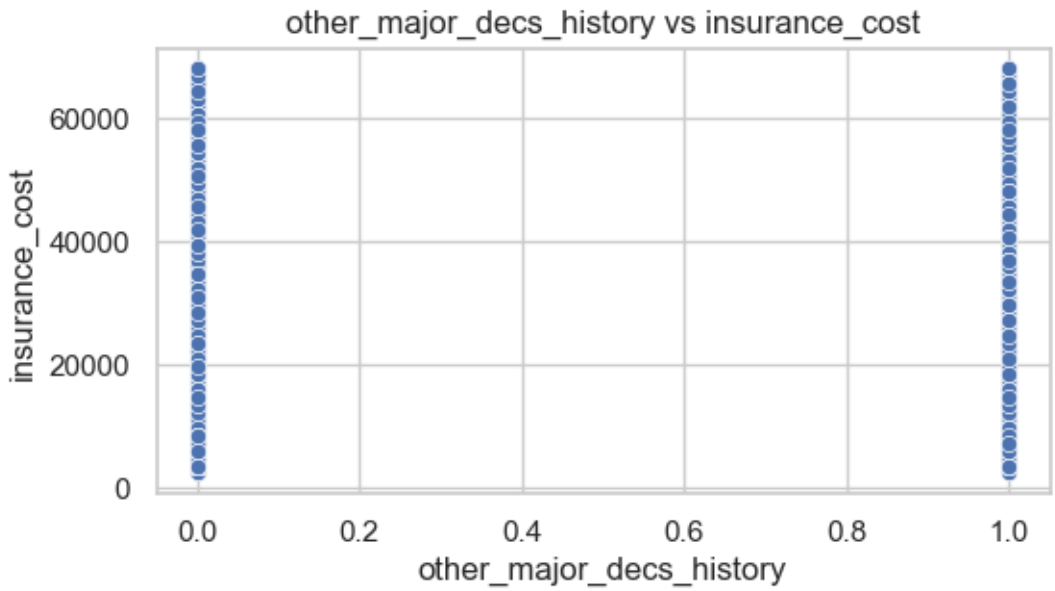

visited_doctor_last_1_year vs insurance_cost
insurance_cost
60000
40000
20000
0
0
2
4
6
8
10
12
visited_doctor_last_1_year

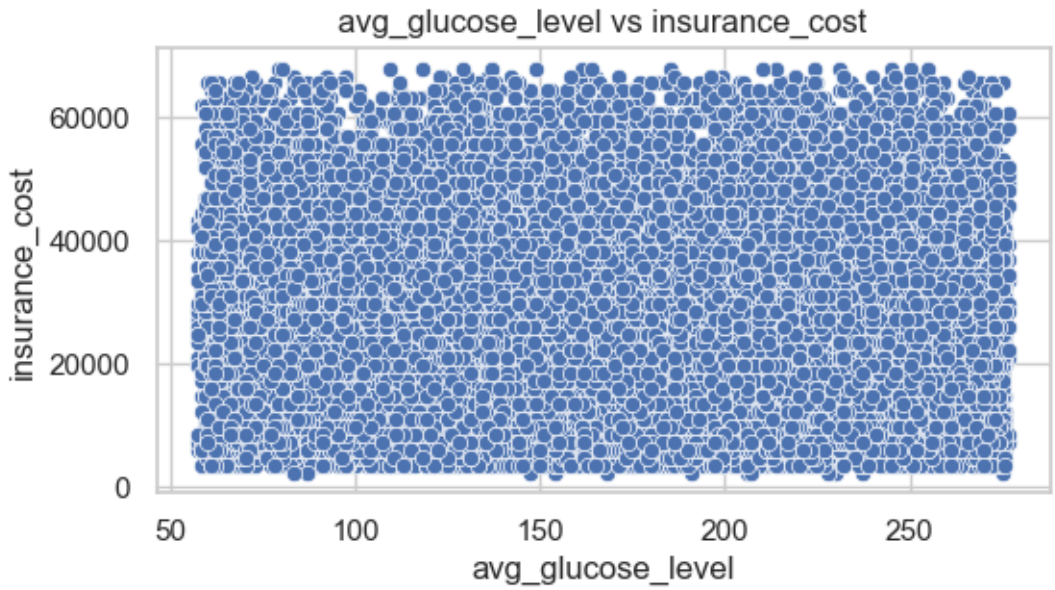

cholesterol_level vs insurance_cost
0.04
0.02
0.00
−0.02
−0.04
−0.04
−0.02
0.00
0.02
0.04

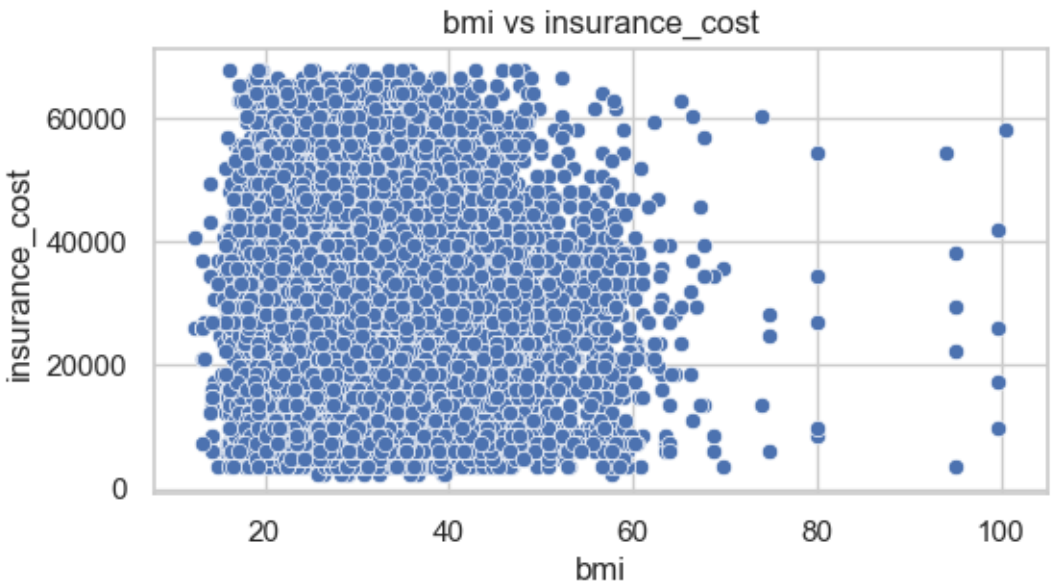

daily_avg_steps vs insurance_cost
insurance_cost
60000
40000
20000
0
2000
4000
6000
8000
10000
daily_avg_steps

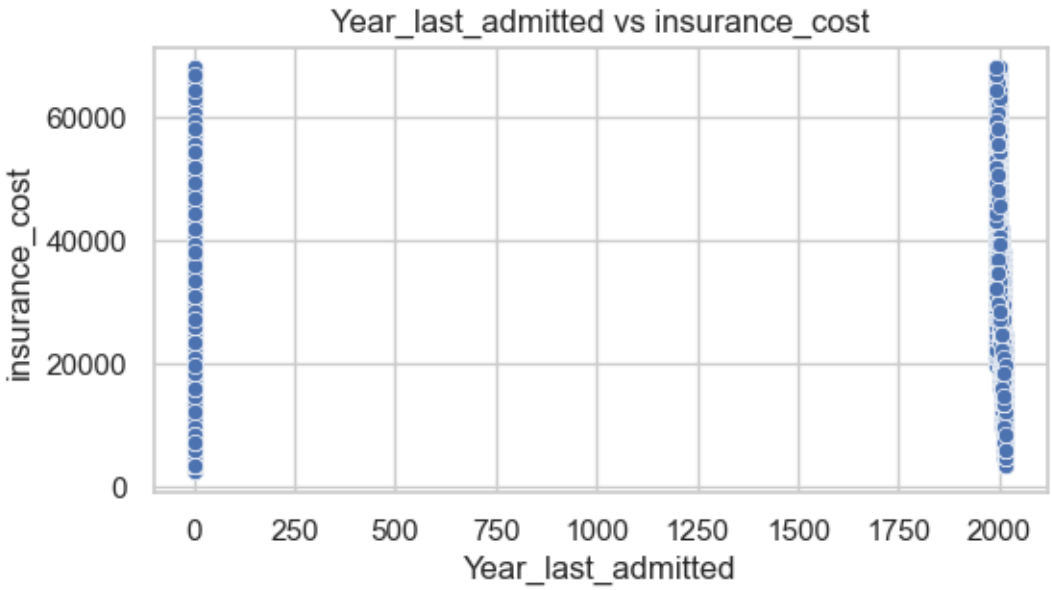

age vs insurance_cost
insurance_cost
60000
40000
20000
0
20
30
40
50
60
70
age

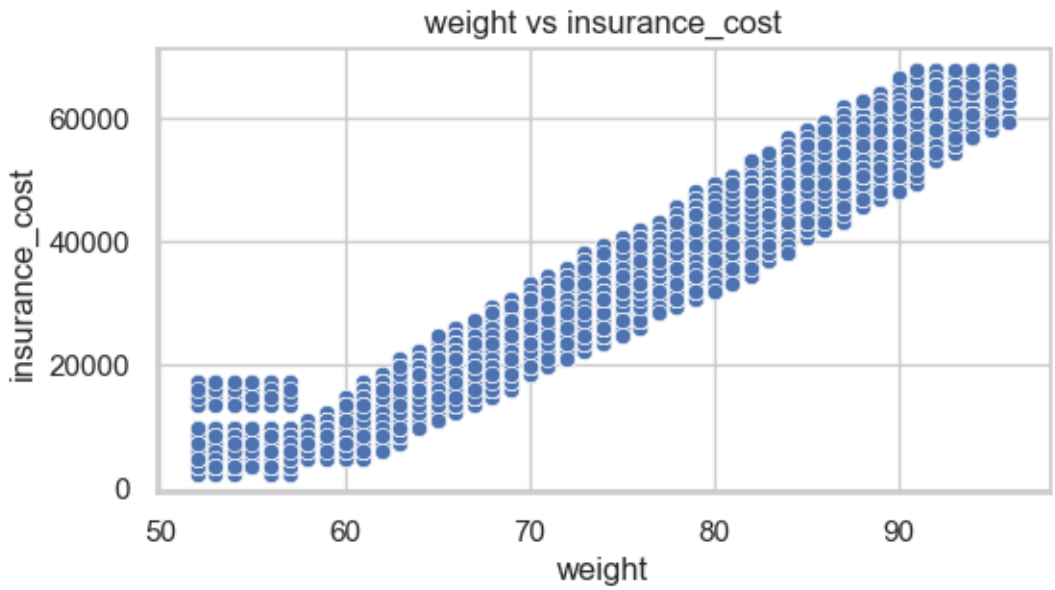

heart_decs_history vs insurance_cost
insurance_cost
60000
40000
20000
0
0.0
0.2
0.4
0.6
0.8
1.0
heart_decs_history

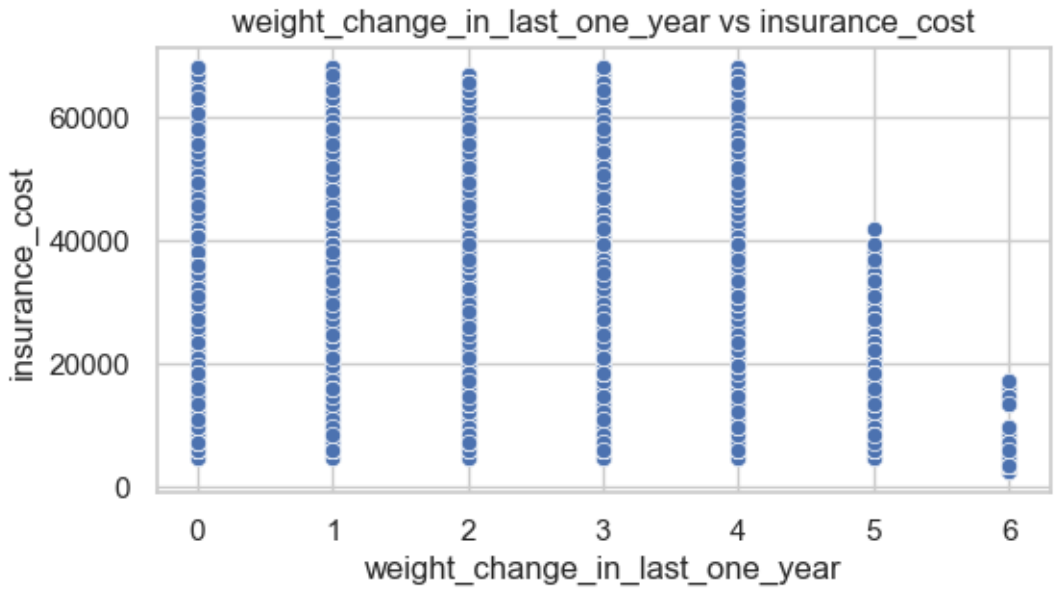

other_major_decs_history vs insurance_cost
insurance_cost
60000
40000
20000
0
0.0
0.2
0.4
0.6
0.8
1.0
other_major_decs_history

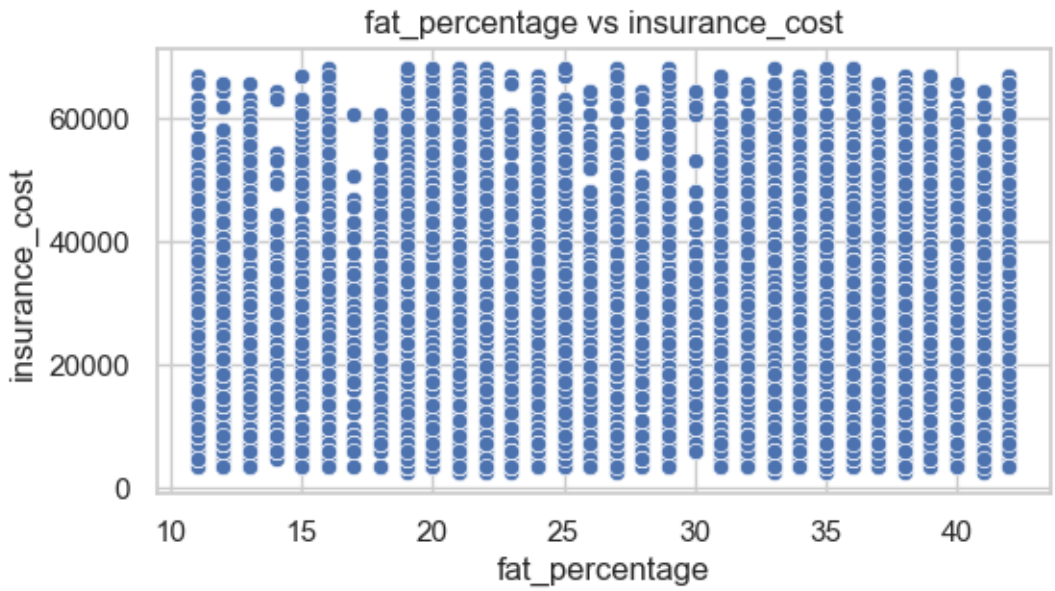

avg_glucose_level vs insurance_cost
insurance_cost
60000
40000
20000
0
50
100
150
200
250
avg_glucose_level

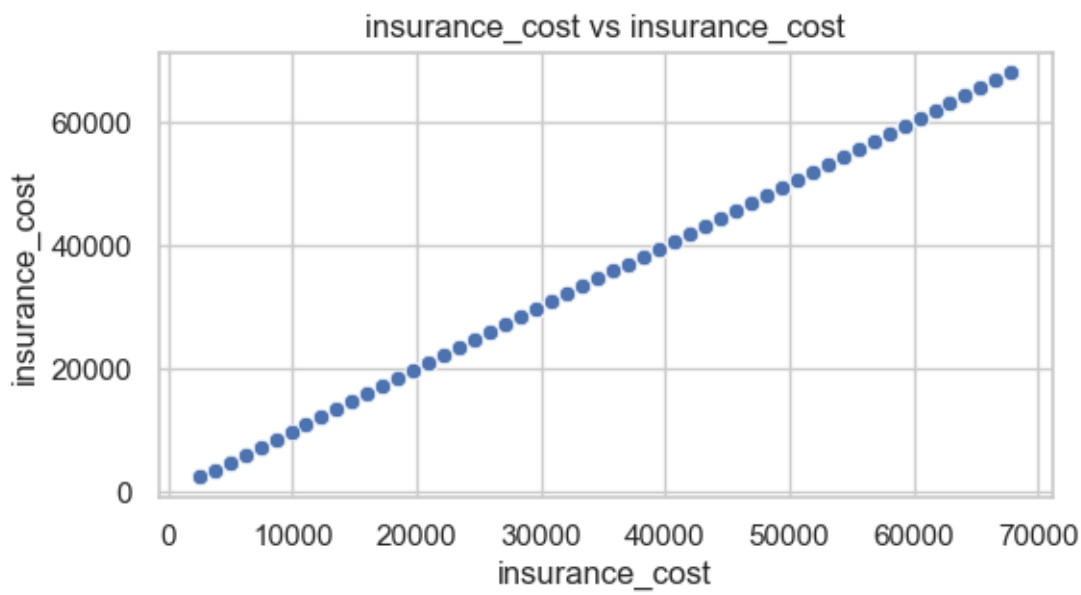

bmi vs insurance_cost
insurance_cost
60000
40000
20000
0
20
40
60
80
100
bmi


Year_last_admitted vs insurance_cost
insurance_cost
60000
40000
20000
0
0
250
500
750
1000
1250
1500
1750
2000
Year_last_admitted

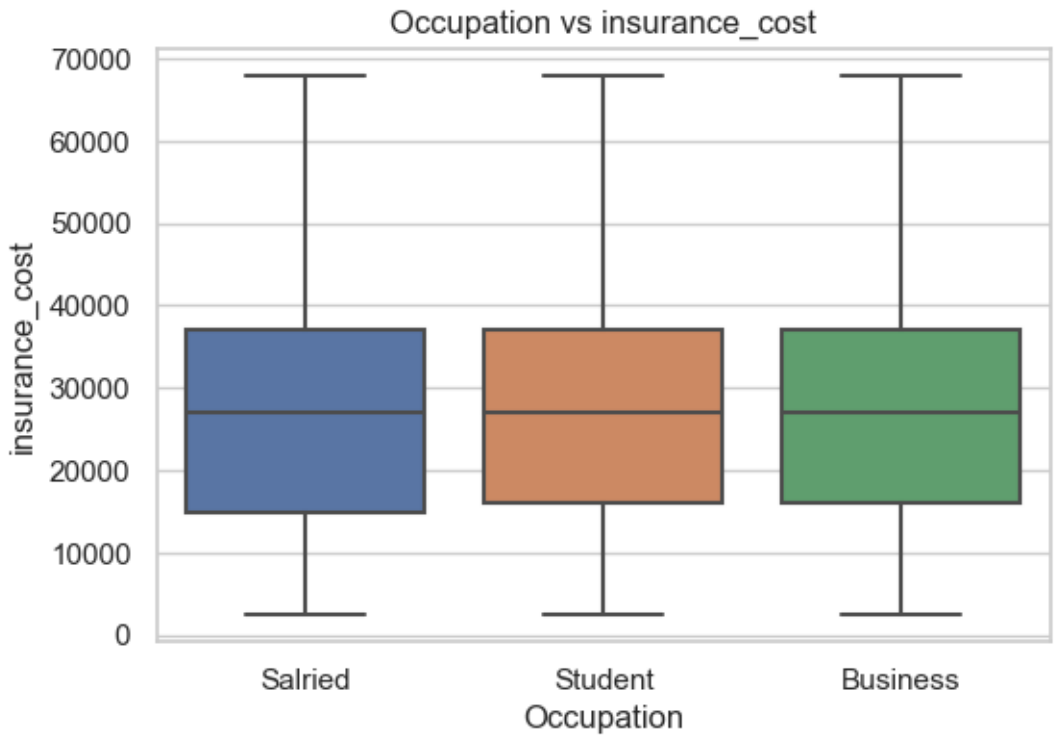

weight vs insurance_cost
insurance_cost
0
20000
40000
60000
50
60
70
80
90
weight


weight_change_in_last_one_year vs insurance_cost
insurance_cost
0
20000
40000
60000
0
1
2
3
4
5
6
weight_change_in_last_one_year





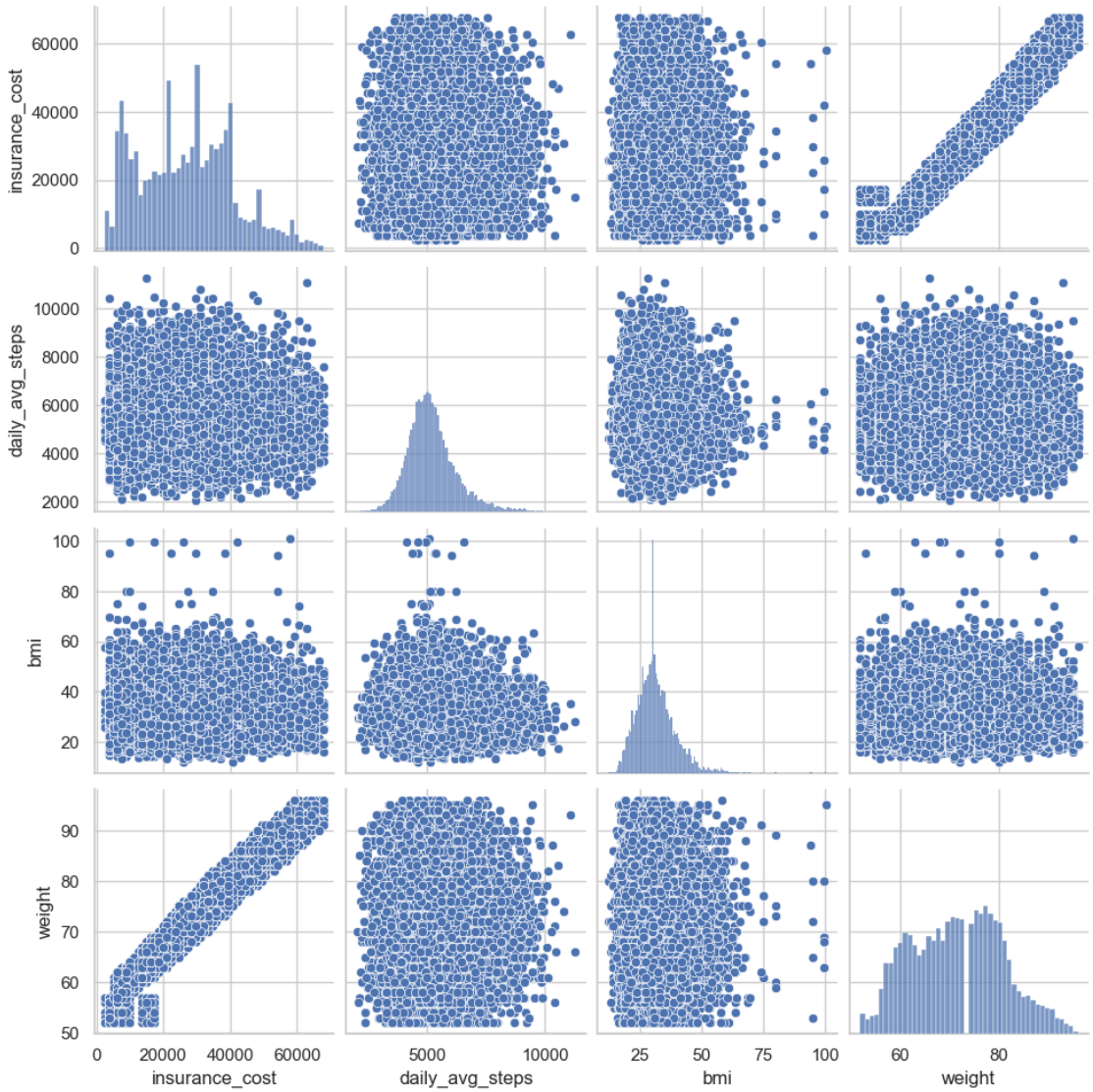
[27]:
```
for col in categorical_cols:
    plt.figure(figsize=(6, 4))
    sns.boxplot(x=col, y='insurance_cost', data=df)
    plt.title(f'{col} vs insurance_cost')
    plt.show()
```



- Median Insurance Costs: The median insurance cost appears to be quite similar for all three occupation groups, with Students potentially having a slightly higher median than the other two.
- Spread of Costs: The overall spread of insurance costs (as indicated by the length of the boxes and whiskers) is fairly similar across the three groups, suggesting comparable levels of variability in costs within each occupation.
- Outliers: All three groups have outliers, represented by the dots above the whiskers. This indicates that there are some individuals within each occupation who have significantly higher insurance costs than the majority.

[29]:
```
sns.pairplot(df[['insurance_cost', 'Occupation', 'daily_avg_steps', 'bmi',␣
↪'weight']])
plt.show()
```

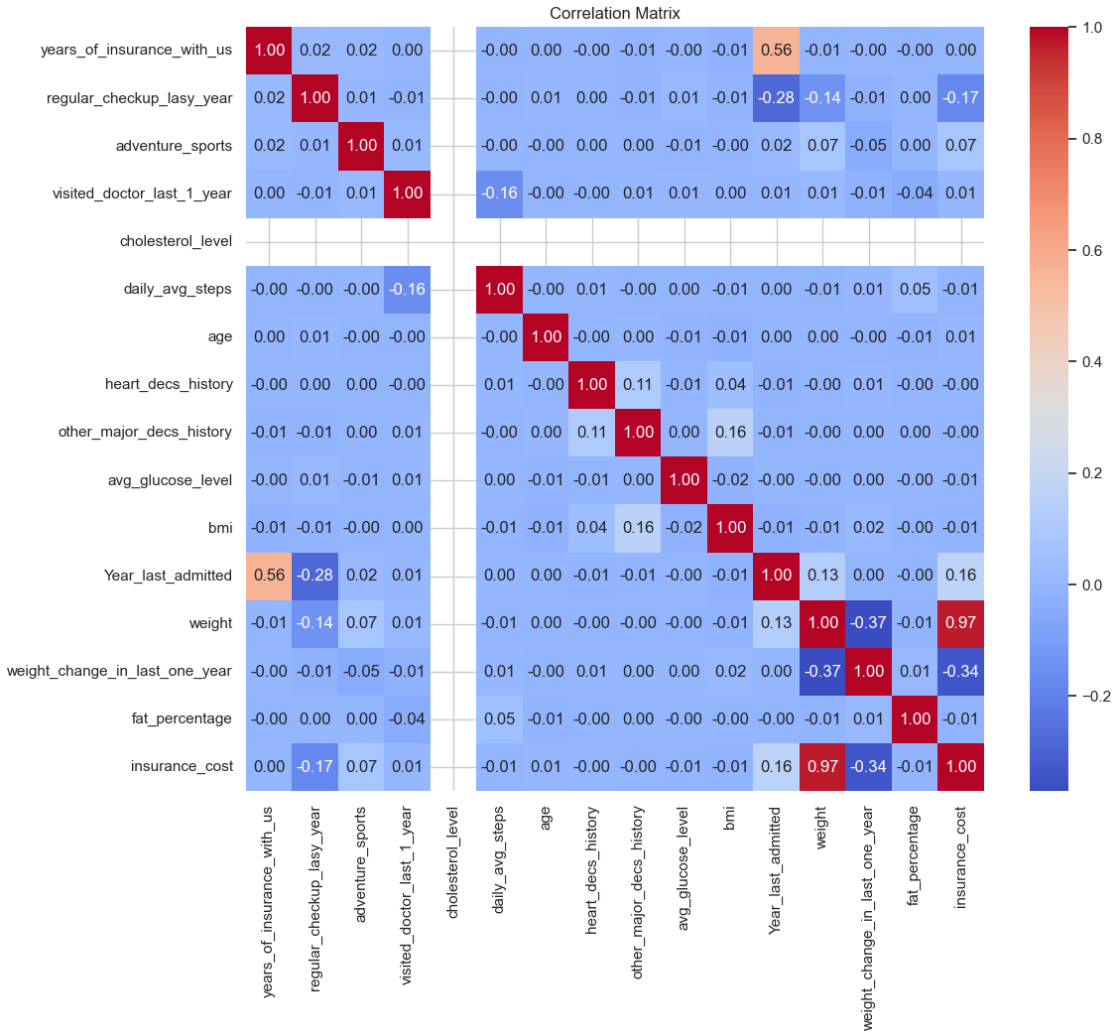
```
C:\Users\Cifer\anaconda3\Lib\site-packages\seaborn\axisgrid.py:118: UserWarning:
The figure layout has changed to tight
  self._figure.tight_layout(*args, **kwargs)
```



- Insurance Cost & Weight: There's a clear positive correlation between 'insurance_cost' and 'weight'. Heavier individuals tend to have higher insurance costs.
- BMI & Weight: A strong positive correlation is evident between 'bmi' (Body Mass Index) and 'weight'.
- Daily Steps & Other Variables: There doesn't appear to be a strong linear relationship between 'daily_avg_steps' and the other variables (insurance cost, BMI, weight) based on the scatterplots. However, a more nuanced relationship might exist.

[30]:
```
sns.set(style="whitegrid")

correlation_matrix = df[numerical_cols].corr()

# Plot the heatmap of the correlation matrix
```

```
plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```



0.1.3 Strong Positive Correlations:

- Years of insurance with us and Year last admitted: This suggests people who have been insured longer are more likely to have had a recent hospital admission.
- Weight and Insurance Cost: Higher weight is strongly correlated with higher insurance costs.
- Weight and Year last admitted: Similar to the first point, this suggests a link between weight and needing hospital care.
- Fat percentage and Weight: This is expected as body fat is a component of weight.

0.1.4 Strong Negative Correlations:

- Weight and Weight change in the last one year: This implies that individuals with higher weight tend to have experienced less * weight loss (or even weight gain) recently.
- Insurance Cost and Weight change in the last one year: This suggests that people with higher insurance costs might be less likely to have lost weight recently, possibly due to health conditions or lifestyle factors.

0.1.5 Other Observations

- Age has a weak positive correlation with Years of insurance with us, which is somewhat expected.
- There's a weak negative correlation between Daily average steps and Insurance cost, hinting at a potential link between activity levels and health risks.

[31]:
```
smoking_status_counts = df['smoking_status'].value_counts(normalize=True) * 100
print("Percentage of individuals by smoking status:\n", smoking_status_counts)
```

```
Percentage of individuals by smoking status:
 smoking_status
2    36.996
0    30.220
1    17.316
3    15.468
Name: proportion, dtype: float64
```

[32]:
```
heart_disease_percentage = (df['heart_decs_history'].
 ↪value_counts(normalize=True) * 100).get(1, 0)
print(f"Percentage of individuals with a history of heart disease:␣
 ↪{heart_disease_percentage:.2f}%")
```

```
Percentage of individuals with a history of heart disease: 5.46%
```

[33]:
```
coverage_percentage = (df['covered_by_any_other_company'].
 ↪value_counts(normalize=True) * 100).get('Y', 0)
print(f"Percentage of individuals covered by other companies:␣
 ↪{coverage_percentage:.2f}%")
```

```
Percentage of individuals covered by other companies: 0.00%
```

[34]:
```
from sklearn.cluster import KMeans

# Applying KMeans clustering
kmeans = KMeans(n_clusters=3, random_state=42)
df['cluster'] = kmeans.fit_predict(df[['age', 'bmi', 'avg_glucose_level']])
```

[36]:
```
# Cluster Profiling
cluster_profiles = df.groupby('cluster').agg(
    age_mean=('age', 'mean'),
    bmi_mean=('bmi', 'mean'),
```

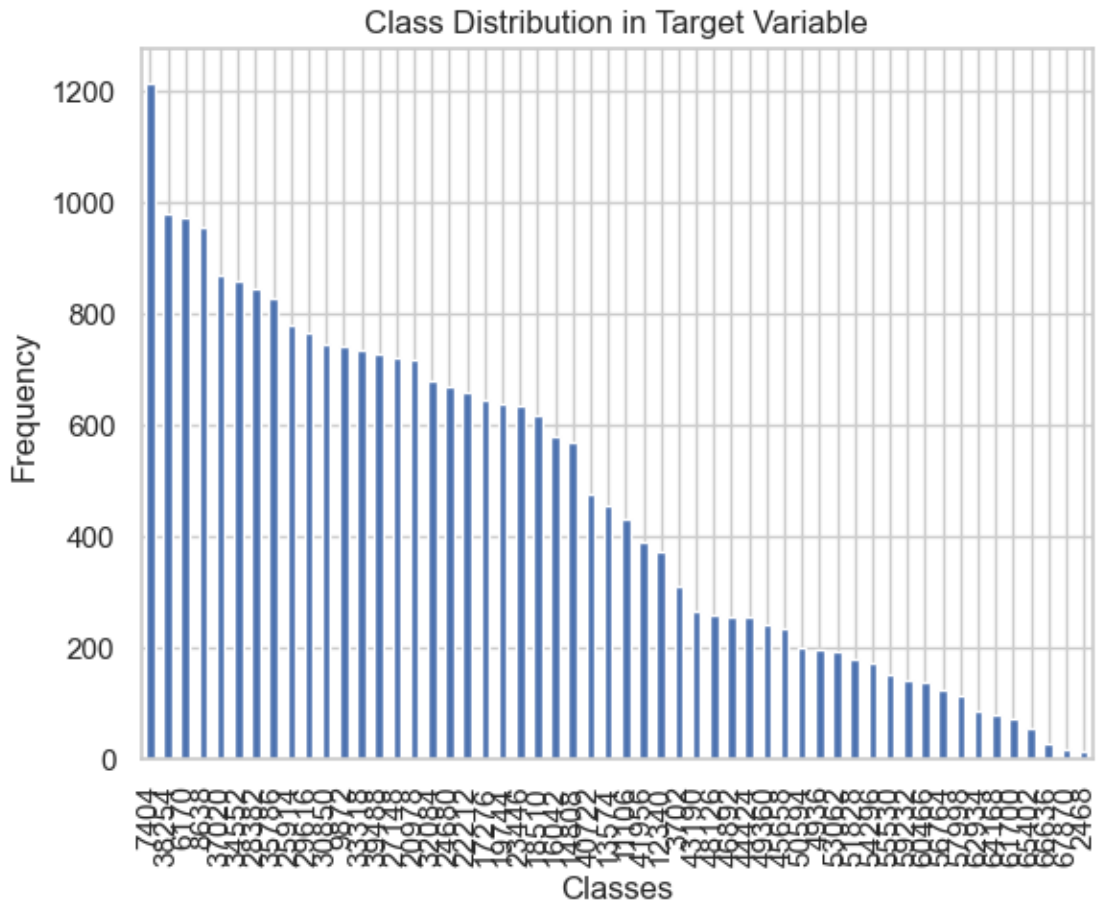
```
    avg_glucose_level_mean=('avg_glucose_level', 'mean'),
    smoking_status_mode=('smoking_status', lambda x: x.mode()[0]),
    Gender_mode=('Gender', lambda x: x.mode()[0])).reset_index()

print("Cluster Profiles:")
print(cluster_profiles)
```

```
Cluster Profiles:
   cluster   age_mean   bmi_mean  avg_glucose_level_mean  smoking_status_mode  \
0        0  44.674709  31.249029              239.223844                    2
1        1  45.192827  31.552851               94.591854                    2
2        2  44.895627  31.276105              166.410606                    2

   Gender_mode
0            1
1            1
2            1
```

0.1.6 Data Imbalance check

[37]:
```
# Data Imbalance Check
for col in df.select_dtypes(include='object').columns:
    print(f"Value counts for {col}:")
    print(df[col].value_counts(normalize=True) * 100)
    print("\n")

# Visualize class imbalance if target variable exists
import matplotlib.pyplot as plt

# Assuming 'insurance_cost' is your target column
df['insurance_cost'].value_counts().plot(kind='bar')
plt.title("Class Distribution in Target Variable")
plt.xlabel("Classes")
plt.ylabel("Frequency")
plt.show()
```

```
Value counts for Occupation:
Occupation
Student     40.676
Business    40.080
Salried     19.244
Name: proportion, dtype: float64
```



- The chart shows a steep decline in the frequency of classes, indicating significant imbalance.
- A few classes dominate the dataset, having significantly higher frequencies compared to others.
- The majority of classes on the right-hand side of the chart have relatively low representation, which can cause issues in model performance and reliability.

[39]:
```
from collections import Counter

# Checking class imbalance for categorical columns
categorical_columns = df.select_dtypes(include='object').columns

# Count the occurrence of each category within each categorical column
imbalance_report = {col: Counter(df[col]) for col in categorical_columns}

# Display imbalance information
imbalance_report
```

[39]:
```
{'Occupation': Counter({'Student': 10169, 'Business': 10020, 'Salried': 4811})}
```

- The data is heavily skewed toward “Student” and “Business” categories, while “Salaried” is

significantly underrepresented.

- Predictive models may become biased toward the majority categories ("Student" and "Business").
- Insights derived from this data might not generalize well to the "Salaried" category due to its lower representation.

[65]:
```
from imblearn.over_sampling import RandomOverSampler

ros = RandomOverSampler(random_state=42)
X_resampled, y_resampled = ros.fit_resample(X, y)
```

[66]:
```
from imblearn.under_sampling import RandomUnderSampler

rus = RandomUnderSampler(random_state=42)
X_resampled, y_resampled = rus.fit_resample(X, y)
```

[72]:
```
# Import necessary libraries
import pandas as pd
from imblearn.over_sampling import SMOTE
from sklearn.impute import SimpleImputer
from collections import Counter

# Define features (X) and target (y)
X = df.drop(columns=['Occupation'])  # Drop the target column
y = df['Occupation']                 # Define the target column

# Separate numerical and categorical columns
numerical_columns = X.select_dtypes(include=['float64', 'int64']).columns
categorical_columns = X.select_dtypes(include=['object']).columns

# Remove columns with all missing values to avoid issues during imputation
numerical_columns_nonmissing = [col for col in numerical_columns if X[col].
 ↪notna().any()]

# Impute only numerical columns that are not fully missing
imputer = SimpleImputer(strategy='mean')
X_numerical_imputed = pd.DataFrame(imputer.
 ↪fit_transform(X[numerical_columns_nonmissing]),
                                   columns=numerical_columns_nonmissing)

# Concatenate imputed numerical data with original categorical data
X_imputed = pd.concat([X_numerical_imputed, X[categorical_columns].
 ↪reset_index(drop=True)], axis=1)

# Apply SMOTE for balancing
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X_imputed, y)
```

```python
# Check the results
print("Resampled class distribution:", Counter(y_resampled))
```

```
Resampled class distribution: Counter({'Salried': 10169, 'Student': 10169,
'Business': 10169})
```

0.2 Business Insights

1. Customer Segmentation

- Demographic Analysis: Understanding the distribution of customer demographics (age, gender, occupation) can help in tailoring marketing strategies and product offerings.
- Behavioral Insights: Identifying patterns in customer behavior (e.g., spending habits, usage frequency) enables personalized marketing and better targeting.

2. Revenue Insights

- Insurance Costs Analysis: Analyzing the relationship between variables (e.g., age, smoking status) and insurance costs can reveal factors that significantly influence pricing, helping adjust premiums more accurately.
- Pricing Strategy: Insights on the distribution of insurance costs can inform pricing strategies to ensure competitiveness while maximizing profitability.

3. Product Development

- Feature Preferences: Insights from customer preferences (e.g., popular insurance products or coverage types) can guide product development and enhancements.
- Gap Analysis: Identifying unmet needs in the customer base can lead to the development of new products or features.

4. Risk Management

- Identification of High-Risk Groups: Analysis of demographic or lifestyle factors (e.g., smoking status, medical history) can help identify high-risk groups, informing underwriting processes and risk mitigation strategies.
- Predictive Modelling: Using EDA insights to develop predictive models that can forecast future claims or customer churn.

5. Marketing Strategy

- Targeted Campaigns: Understanding the characteristics of customers who are more likely to purchase certain products can lead to more effective marketing campaigns.
- Channel Optimization: Identifying which customer segments respond best to different marketing channels (e.g., email, social media) can improve marketing ROI.

6. Operational Efficiency

- Process Improvement: Insights into customer feedback or complaints can highlight areas for operational improvement, reducing costs and enhancing service delivery.
- Resource Allocation: Understanding customer needs and preferences can guide resource allocation, ensuring that high-demand services are adequately supported.

7. Trend Analysis

- Market Trends: Analyzing historical data to identify trends in customer behavior, insurance costs, or market conditions can inform long-term strategic planning.
- Seasonal Variations: Identifying seasonal patterns in claims or customer engagement can help in planning marketing campaigns and resource allocation.

8. Customer Satisfaction

- Feedback Analysis: Analyzing customer feedback and survey results can help improve service offerings and customer satisfaction.
- Churn Prediction: Understanding the factors contributing to customer churn can lead to strategies to improve retention.

[ ]:

```

```

0.3 Model Building

0.3.1 Linear Regression

[78]:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.impute import SimpleImputer
import statsmodels.api as sm

# Assuming 'insurance_cost' is the target variable
X = df.drop(columns=['insurance_cost'])
y = df['insurance_cost']

# Drop columns with all missing values
X = X.dropna(axis=1, how='all')

# Convert categorical columns to numerical using one-hot encoding
X = pd.get_dummies(X, drop_first=True)

# Impute missing values for numerical columns
imputer = SimpleImputer(strategy='mean')
X_imputed = pd.DataFrame(imputer.fit_transform(X), columns=X.columns)

# Ensure the target variable y is numeric
y = pd.to_numeric(y, errors='coerce').fillna(y.mean())  # Fill any NaN in y␣
↪with its mean

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_imputed, y, test_size=0.
↪2, random_state=42)

# Adding a constant to the model for the intercept
X_train_const = sm.add_constant(X_train)
```

```python
X_test_const = sm.add_constant(X_test)

# Fitting the linear regression model using statsmodels for detailed output
model = sm.OLS(y_train, X_train_const).fit()

# Predicting the test set
y_pred = model.predict(X_test_const)

# Calculating performance metrics
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

# Detailed model summary
model_summary = model.summary()

print("Mean Squared Error:", mse)
print("R-squared:", r2)
print(model_summary)
```

```
Mean Squared Error: 11231678.8994767
R-squared: 0.9448710858029175
                            OLS Regression Results
==============================================================================
Dep. Variable:         insurance_cost   R-squared:                       0.946
Model:                            OLS   Adj. R-squared:                  0.946
Method:                 Least Squares   F-statistic:                 1.509e+04
Date:                Fri, 08 Nov 2024   Prob (F-statistic):               0.00
Time:                        04:04:53   Log-Likelihood:            -1.9068e+05
No. Observations:               20000   AIC:                         3.814e+05
Df Residuals:                   19976   BIC:                         3.816e+05
Df Model:                          23
Covariance Type:            nonrobust
==================================================================================
==================
                                     coef    std err          t      P>|t|
[0.025      0.975]
----------------------------------------------------------------------------------
------------------
const                          -7.809e+04    341.023   -228.997      0.000
-7.88e+04   -7.74e+04
years_of_insurance_with_us       -81.8038     11.454     -7.142      0.000
-104.255     -59.352
regular_checkup_lasy_year       -327.2864     21.467    -15.246      0.000
-369.364    -285.209
adventure_sports               -1820.4194    184.490     -9.867      0.000
-2182.034   -1458.805
visited_doctor_last_1_year       -52.2901     21.184     -2.468      0.014
-93.812     -10.768
```

```
daily_avg_steps                  -0.0197      0.023     -0.854      0.393
-0.065       0.026
age                               3.8800      1.473      2.634      0.008
0.992       6.768
heart_decs_history              285.4394    105.101      2.716      0.007
79.432     491.447
other_major_decs_history       -1219.4498    127.543     -9.561      0.000
-1469.445    -969.454
Gender                            25.0335     54.632      0.458      0.647
-82.049     132.116
avg_glucose_level                  0.1582      0.378      0.419      0.675
-0.582       0.898
bmi                               -3.7975      3.318     -1.144      0.252
-10.301       2.706
smoking_status                    -7.2621     23.104     -0.314      0.753
-52.548      38.024
Year_last_admitted                 0.3384      0.031     10.912      0.000
0.278       0.399
Location                           7.0703      5.499      1.286      0.199
-3.708      17.848
weight                          1453.9876      3.685    394.567      0.000
1446.765    1461.211
covered_by_any_other_company    1197.5240     53.904     22.216      0.000
1091.869    1303.179
Alcohol                           25.4340     35.110      0.724      0.469
-43.384      94.252
exercise                          22.7251     37.150      0.612      0.541
-50.093      95.543
weight_change_in_last_one_year   201.5977     15.465     13.036      0.000
171.285     231.910
fat_percentage                    -1.5714      2.858     -0.550      0.582
-7.173       4.030
cluster                          934.8454     73.978     12.637      0.000
789.843    1079.848
Occupation_Salried                87.1951     68.538      1.272      0.203
-47.145     221.535
Occupation_Student                14.7731     53.159      0.278      0.781
-89.423     118.970
==============================================================================
Omnibus:                      430.740   Durbin-Watson:                   1.970
Prob(Omnibus):                  0.000   Jarque-Bera (JB):              471.722
Skew:                           0.341   Prob(JB):                    3.69e-103
Kurtosis:                       3.318   Cond. No.                     8.09e+04
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
```

```
[2] The condition number is large, 8.09e+04. This might indicate that there are
strong multicollinearity or other numerical problems.
```

- Mean Squared Error: 11231678.8994767. This represents the average squared difference between the predicted and actual insurance costs.
- R-squared: 0.9448710858029175. This indicates that approximately 94.5% of the variance in insurance cost is explained by the model.
- Significant Predictors (p-value < 0.05): years_of_insurance_with_us: Each additional year with the insurance provider reduces costs by 81.80 units on average, likely reflecting loyalty benefits. regular_checkup_lasy_year: Regular checkups correlate with lower insurance costs, reducing it by 327.29 units per instance. adventure_sports: Participation in adventure sports decreases costs by 1,820.42 units, possibly due to insurers' preferences or incentive programs. weight: Shows a strong positive correlation with insurance cost, increasing it by 1,453.99 units per unit increase in weight. covered_by_any_other_company: Being covered by another insurer increases costs by 1,197.52 units, possibly indicating higher perceived risk. Year_last_admitted: Later years for the last hospital admission slightly increase costs. cluster: Belonging to certain clusters increases costs by 934.85 units, highlighting cluster-based risk assessment.
- Non-significant Predictors (p-value > 0.05): daily_avg_steps, Gender, avg_glucose_level, bmi, smoking_status, Location, Alcohol, exercise, fat_percentage, and some occupation categories (Salried, Student). These variables have high p-values, indicating they might not significantly impact insurance_cost in this model and could potentially be removed to simplify the model.

[81]:
```python
# For evaluating on training data
y_train_pred = model.predict(X_train_const)
mse_train = mean_squared_error(y_train, y_train_pred)
r2_train = r2_score(y_train, y_train_pred)

# For evaluating on test data (already in the notebook)
y_test_pred = model.predict(X_test_const)
mse_test = mean_squared_error(y_test, y_test_pred)
r2_test = r2_score(y_test, y_test_pred)

# Print results
print(f"Training MSE: {mse_train:.4f}, Training R²: {r2_train:.4f}")
print(f"Test MSE: {mse_test:.4f}, Test R²: {r2_test:.4f}")

# Comment on overfitting or underfitting
if abs(r2_train - r2_test) > 0.1:  # Adjust threshold based on requirements
    if r2_train > r2_test:
        print("The model may be overfitting.")
    else:
        print("The model may be underfitting.")
else:
    print("The model has a good fit.")
```

```
Training MSE: 11183890.7307, Training R²: 0.9456
Test MSE: 11231678.8995, Test R²: 0.9449
The model has a good fit.
```

0.3.2 Decision Tree

[82]:
```python
from sklearn.tree import DecisionTreeRegressor

# Initialize the Decision Tree regression model
tree_model = DecisionTreeRegressor(max_depth=5, random_state=42)

# Fit the model
tree_model.fit(X_train, y_train)

# Predict on the test set
y_pred_tree = tree_model.predict(X_test)

# Evaluate the model
mse_tree = mean_squared_error(y_test, y_pred_tree)
r2_tree = r2_score(y_test, y_pred_tree)

print("Decision Tree Regression MSE:", mse_tree)
print("Decision Tree Regression R-squared:", r2_tree)
```

```
Decision Tree Regression MSE: 9770814.433026247
Decision Tree Regression R-squared: 0.9520415072995885
```

[93]:
```python
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
↪random_state=42)

# Initialize and fit the Decision Tree model
tree_model = DecisionTreeRegressor(max_depth=5, random_state=42)
tree_model.fit(X_train, y_train)

# Predict on both training and testing data
y_train_pred_tree = tree_model.predict(X_train)
y_test_pred_tree = tree_model.predict(X_test)

# Calculate MSE and R-squared for both sets
mse_train_tree = mean_squared_error(y_train, y_train_pred_tree)
r2_train_tree = r2_score(y_train, y_train_pred_tree)

mse_test_tree = mean_squared_error(y_test, y_test_pred_tree)
r2_test_tree = r2_score(y_test, y_test_pred_tree)

# Print results
print(f"Training MSE: {mse_train_tree:.4f}, Training R²: {r2_train_tree:.4f}")
```

```
print(f"Test MSE: {mse_test_tree:.4f}, Test R²: {r2_test_tree:.4f}")

# Overfitting/Underfitting Analysis
if abs(r2_train_tree - r2_test_tree) > 0.1:  # Adjust threshold as needed
    if r2_train_tree > r2_test_tree:
        print("The Decision Tree model may be overfitting.")
    else:
        print("The Decision Tree model may be underfitting.")
else:
    print("The Decision Tree model has a good fit.")
```

```
Training MSE: 9899245.3248, Training R²: 0.9518
Test MSE: 9770814.4330, Test R²: 0.9520
The Decision Tree model has a good fit.
```

0.3.3 Random Forest

[89]:
```
# Assuming `df` is your DataFrame and includes 'insurance_cost' as the target␣
↪variable
# Convert categorical columns to dummy variables
df = pd.get_dummies(df, drop_first=True)

# Prepare feature and target variables
X = df.drop(columns=['insurance_cost'])
y = df['insurance_cost']

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
↪random_state=42)

# Initialize and train the Random Forest Regressor
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# Predict on the test set
y_pred_rf = rf_model.predict(X_test)

# Performance metrics
mse_rf = mean_squared_error(y_test, y_pred_rf)
r2_rf = r2_score(y_test, y_pred_rf)

# Print performance metrics
print(f"Mean Squared Error (MSE): {mse_rf}")
print(f"R-squared (R²): {r2_rf}")

# Feature importances
feature_importances = rf_model.feature_importances_
```

```
features = X.columns

# Create a DataFrame for feature importances
feature_importance_df = pd.DataFrame({
    'Feature': features,
    'Importance': feature_importances
}).sort_values(by='Importance', ascending=False)

# Display the feature importances
print(feature_importance_df)
```

```
Mean Squared Error (MSE): 8946889.10497872
R-squared (R²): 0.9560856140730515
                           Feature  Importance
15                          weight    0.951415
5                  daily_avg_steps    0.005060
11                             bmi    0.004974
10               avg_glucose_level    0.004913
6                              age    0.004403
13              Year_last_admitted    0.004331
20                  fat_percentage    0.003577
14                        Location    0.003231
19  weight_change_in_last_one_year    0.002716
0       years_of_insurance_with_us    0.002368
16    covered_by_any_other_company    0.002361
1        regular_checkup_lasy_year    0.002125
3       visited_doctor_last_1_year    0.001572
21                         cluster    0.001384
12                  smoking_status    0.001331
18                        exercise    0.001017
17                         Alcohol    0.000931
23              Occupation_Student    0.000584
9                           Gender    0.000522
22              Occupation_Salried    0.000431
7               heart_decs_history    0.000299
8         other_major_decs_history    0.000289
2                 adventure_sports    0.000164
4                cholesterol_level    0.000000
```

[94]:
```
# For evaluating on training data
y_train_pred_rf = rf_model.predict(X_train)
mse_train_rf = mean_squared_error(y_train, y_train_pred_rf)
r2_train_rf = r2_score(y_train, y_train_pred_rf)

# For evaluating on test data (already computed)
mse_test_rf = mse_rf
r2_test_rf = r2_rf
```

```
# Print results
print(f"Training MSE: {mse_train_rf:.4f}, Training R²: {r2_train_rf:.4f}")
print(f"Test MSE: {mse_test_rf:.4f}, Test R²: {r2_test_rf:.4f}")

# Comment on overfitting or underfitting
if abs(r2_train_rf - r2_test_rf) > 0.1:  # Adjust threshold based on␣
 ↪requirements
    if r2_train_rf > r2_test_rf:
        print("The Random Forest model may be overfitting.")
    else:
        print("The Random Forest model may be underfitting.")
else:
    print("The Random Forest model has a good fit.")
```

```
Training MSE: 1288383.1040, Training R²: 0.9937
Test MSE: 8946889.1050, Test R²: 0.9561
The Random Forest model has a good fit.
```

0.3.4 K- Nearest Neighbour (KNN)

[96]:
```
from sklearn.impute import SimpleImputer
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Initialize an imputer to fill missing values with the median
imputer = SimpleImputer(strategy='median')

# Apply the imputer to the training and test sets
X_train_imputed = imputer.fit_transform(X_train)
X_test_imputed = imputer.transform(X_test)

# Initialize and fit the K-Nearest Neighbors regression model
knn_model = KNeighborsRegressor(n_neighbors=5)
knn_model.fit(X_train_imputed, y_train)

# Predict on the test set
y_pred_knn = knn_model.predict(X_test_imputed)

# Evaluate the model's performance
mse_knn = mean_squared_error(y_test, y_pred_knn)
r2_knn = r2_score(y_test, y_pred_knn)

# Print the performance metrics
print("KNN Regression MSE:", mse_knn)
print("KNN Regression R-squared:", r2_knn)
```

```
KNN Regression MSE: 94157631.58617601
```

```
KNN Regression R-squared: 0.5378422015824673
```

```
C:\Users\Cifer\anaconda3\Lib\site-packages\sklearn\impute\_base.py:598:
UserWarning: Skipping features without any observed values:
['cholesterol_level']. At least one non-missing value is needed for imputation
with strategy='median'.
  warnings.warn(
C:\Users\Cifer\anaconda3\Lib\site-packages\sklearn\impute\_base.py:598:
UserWarning: Skipping features without any observed values:
['cholesterol_level']. At least one non-missing value is needed for imputation
with strategy='median'.
  warnings.warn(
```

[97]:
```
# For evaluating on training data
y_train_pred_knn = knn_model.predict(X_train_imputed)
mse_train_knn = mean_squared_error(y_train, y_train_pred_knn)
r2_train_knn = r2_score(y_train, y_train_pred_knn)

# For evaluating on test data (already computed)
mse_test_knn = mse_knn
r2_test_knn = r2_knn

# Print results
print(f"Training MSE: {mse_train_knn:.4f}, Training R²: {r2_train_knn:.4f}")
print(f"Test MSE: {mse_test_knn:.4f}, Test R²: {r2_test_knn:.4f}")

# Comment on overfitting or underfitting
if abs(r2_train_knn - r2_test_knn) > 0.1:  # Adjust threshold based on␣
 ↪requirements
    if r2_train_knn > r2_test_knn:
        print("The KNN model may be overfitting.")
    else:
        print("The KNN model may be underfitting.")
else:
    print("The KNN model has a good fit.")
```

```
Training MSE: 62114051.7103, Training R²: 0.6978
Test MSE: 94157631.5862, Test R²: 0.5378
The KNN model may be overfitting.
```

0.3.5 Gradient Boosting

[101]:
```
from sklearn.impute import SimpleImputer
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
import pandas as pd
```

```
# Assuming 'data_cleaned' is the preprocessed dataset
X = df.drop(columns=['insurance_cost'])
y = df['insurance_cost']

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
↪random_state=42)

# Initialize an imputer to fill missing values with the median
imputer = SimpleImputer(strategy='median')

# Apply the imputer to the training and test sets
X_train_imputed = imputer.fit_transform(X_train)
X_test_imputed = imputer.transform(X_test)

# Initialize and train the Gradient Boosting Regressor model
gbr_model = GradientBoostingRegressor(n_estimators=100, random_state=42)
gbr_model.fit(X_train_imputed, y_train)

# Predict on the test set
y_pred_gbr = gbr_model.predict(X_test_imputed)

# Evaluate the model
mse_gbr = mean_squared_error(y_test, y_pred_gbr)
r2_gbr = r2_score(y_test, y_pred_gbr)

# Print performance metrics
print("Gradient Boosting Regression MSE:", mse_gbr)
print("Gradient Boosting Regression R-squared:", r2_gbr)
```

```
C:\Users\Cifer\anaconda3\Lib\site-packages\sklearn\impute\_base.py:598:
UserWarning: Skipping features without any observed values:
['cholesterol_level']. At least one non-missing value is needed for imputation
with strategy='median'.
  warnings.warn(
C:\Users\Cifer\anaconda3\Lib\site-packages\sklearn\impute\_base.py:598:
UserWarning: Skipping features without any observed values:
['cholesterol_level']. At least one non-missing value is needed for imputation
with strategy='median'.
  warnings.warn(

Gradient Boosting Regression MSE: 8627507.217888962
Gradient Boosting Regression R-squared: 0.9576532494022891
```

[102]:
```
# For evaluating on training data
y_train_pred_gbr = gbr_model.predict(X_train_imputed)
mse_train_gbr = mean_squared_error(y_train, y_train_pred_gbr)
r2_train_gbr = r2_score(y_train, y_train_pred_gbr)
```

```
# For evaluating on test data (already computed)
mse_test_gbr = mse_gbr
r2_test_gbr = r2_gbr

# Print results
print(f"Training MSE: {mse_train_gbr:.4f}, Training R²: {r2_train_gbr:.4f}")
print(f"Test MSE: {mse_test_gbr:.4f}, Test R²: {r2_test_gbr:.4f}")

# Comment on overfitting or underfitting
if abs(r2_train_gbr - r2_test_gbr) > 0.1:  # Adjust threshold based on␣
 ↪requirements
    if r2_train_gbr > r2_test_gbr:
        print("The Gradient Boosting model may be overfitting.")
    else:
        print("The Gradient Boosting model may be underfitting.")
else:
    print("The Gradient Boosting model has a good fit.")
```

```
Training MSE: 8609183.6212, Training R²: 0.9581
Test MSE: 8627507.2179, Test R²: 0.9577
The Gradient Boosting model has a good fit.
```

0.3.6 Intepretation of the model

- Best Fit Model: The Gradient Boosting model provides the best fit. It has a high R^2 score on both the training and test sets (around 0.957) with minimal difference between them, indicating a well-generalized model with no signs of overfitting or underfitting. Its MSE values are also among the lowest.
- Best Overall Model: The Gradient Boosting model also stands out as the best overall model, as it achieves a high R^2 score with a good balance between training and test performance, indicating it has learned the patterns effectively without overfitting.
- Recommendation: The Gradient Boosting model should be selected as the best model for this dataset due to its superior performance metrics and balanced fit.

0.4 Model Tuning

0.4.1 Linear Regression

[104]:
```
from sklearn.impute import SimpleImputer
from sklearn.linear_model import Ridge, Lasso
from sklearn.model_selection import GridSearchCV

# Impute missing values in X_train
imputer = SimpleImputer(strategy='median')
X_train_imputed = imputer.fit_transform(X_train)
X_test_imputed = imputer.transform(X_test)
```

```
# Ridge Regression with GridSearchCV
ridge = Ridge()
ridge_params = {'alpha': [0.1, 1, 10, 100]}
ridge_grid = GridSearchCV(ridge, ridge_params, cv=5, scoring='r2')
ridge_grid.fit(X_train_imputed, y_train)
print("Best Ridge R2:", ridge_grid.best_score_)
print("Best Ridge Alpha:", ridge_grid.best_params_)

# Lasso Regression with GridSearchCV
lasso = Lasso()
lasso_params = {'alpha': [0.001, 0.01, 0.1, 1]}
lasso_grid = GridSearchCV(lasso, lasso_params, cv=5, scoring='r2')
lasso_grid.fit(X_train_imputed, y_train)
print("Best Lasso R2:", lasso_grid.best_score_)
print("Best Lasso Alpha:", lasso_grid.best_params_)
```

```
C:\Users\Cifer\anaconda3\Lib\site-packages\sklearn\impute\_base.py:598:
UserWarning: Skipping features without any observed values:
['cholesterol_level']. At least one non-missing value is needed for imputation
with strategy='median'.
  warnings.warn(
C:\Users\Cifer\anaconda3\Lib\site-packages\sklearn\impute\_base.py:598:
UserWarning: Skipping features without any observed values:
['cholesterol_level']. At least one non-missing value is needed for imputation
with strategy='median'.
  warnings.warn(

Best Ridge R2: 0.9454089084747057
Best Ridge Alpha: {'alpha': 1}
Best Lasso R2: 0.9454093571054338
Best Lasso Alpha: {'alpha': 1}
```

0.4.2 Decision Tree

[105]:
```
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import GridSearchCV

# Define the Decision Tree and parameters for tuning
tree_model = DecisionTreeRegressor(random_state=42)
param_grid = {
    'max_depth': [3, 5, 10],
    'min_samples_split': [2, 10, 20],
    'min_samples_leaf': [1, 5, 10]
}

# Perform GridSearch to find the best hyperparameters
tree_grid = GridSearchCV(tree_model, param_grid, cv=5, scoring='r2')
```

```
tree_grid.fit(X_train, y_train)

# Best parameters and performance
print("Best Decision Tree R2:", tree_grid.best_score_)
print("Best Decision Tree Params:", tree_grid.best_params_)
```

```
Best Decision Tree R2: 0.9512596134903915
Best Decision Tree Params: {'max_depth': 5, 'min_samples_leaf': 1,
'min_samples_split': 2}
```

0.4.3 Random Forest

[106]:
```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import GridSearchCV

# Define the Random Forest and parameters for tuning
rf_model = RandomForestRegressor(random_state=42)
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [5, 10, 15],
    'min_samples_split': [2, 10, 20],
    'max_features': ['auto', 'sqrt']
}

# Perform GridSearch to find the best hyperparameters
rf_grid = GridSearchCV(rf_model, param_grid, cv=5, scoring='r2')
rf_grid.fit(X_train, y_train)

# Best parameters and performance
print("Best Random Forest R2:", rf_grid.best_score_)
print("Best Random Forest Params:", rf_grid.best_params_)
```

```
C:\Users\Cifer\anaconda3\Lib\site-
packages\sklearn\model_selection\_validation.py:540: FitFailedWarning:
135 fits failed out of a total of 270.
The score on these train-test partitions for these parameters will be set to
nan.
If these failures are not expected, you can try to debug them by setting
error_score='raise'.

Below are more details about the failures:
--------------------------------------------------------------------------------
135 fits failed with the following error:
Traceback (most recent call last):
  File "C:\Users\Cifer\anaconda3\Lib\site-
packages\sklearn\model_selection\_validation.py", line 888, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
  File "C:\Users\Cifer\anaconda3\Lib\site-packages\sklearn\base.py", line 1466,
```

```
in wrapper
    estimator._validate_params()
  File "C:\Users\Cifer\anaconda3\Lib\site-packages\sklearn\base.py", line 666,
in _validate_params
    validate_parameter_constraints(
  File "C:\Users\Cifer\anaconda3\Lib\site-
packages\sklearn\utils\_param_validation.py", line 95, in
validate_parameter_constraints
    raise InvalidParameterError(
sklearn.utils._param_validation.InvalidParameterError: The 'max_features'
parameter of RandomForestRegressor must be an int in the range [1, inf), a float
in the range (0.0, 1.0], a str among {'log2', 'sqrt'} or None. Got 'auto'
instead.

  warnings.warn(some_fits_failed_message, FitFailedWarning)
C:\Users\Cifer\anaconda3\Lib\site-
packages\sklearn\model_selection\_search.py:1103: UserWarning: One or more of
the test scores are non-finite: [       nan        nan        nan        nan
nan        nan
        nan        nan        nan 0.80827156 0.81439481 0.81071454
 0.81152985 0.81667379 0.81264734 0.8093864  0.81731284 0.81221592
        nan        nan        nan        nan        nan        nan
        nan        nan        nan 0.92016112 0.91939842 0.9188099
 0.91911712 0.91922246 0.91850965 0.91618013 0.91786524 0.91658269
        nan        nan        nan        nan        nan        nan
        nan        nan        nan 0.94126287 0.94197146 0.94248623
 0.9412876  0.94186831 0.94147677 0.93858759 0.93930704 0.93877582]
  warnings.warn(
```

```
Best Random Forest R2: 0.9424862329539284
Best Random Forest Params: {'max_depth': 15, 'max_features': 'sqrt',
'min_samples_split': 2, 'n_estimators': 300}
```

0.4.4 KNN

[108]:
```python
from sklearn.impute import SimpleImputer
from sklearn.neighbors import KNeighborsRegressor
from sklearn.model_selection import GridSearchCV

# Impute missing values in X_train
imputer = SimpleImputer(strategy='median')
X_train_imputed = imputer.fit_transform(X_train)
X_test_imputed = imputer.transform(X_test)

# Define the KNN model and parameters for tuning
knn_model = KNeighborsRegressor()
param_grid = {
    'n_neighbors': [3, 5, 7, 9],
```

```
    'weights': ['uniform', 'distance'],
    'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute']
}

# Perform GridSearch to find the best hyperparameters
knn_grid = GridSearchCV(knn_model, param_grid, cv=5, scoring='r2')
knn_grid.fit(X_train_imputed, y_train)

# Best parameters and performance
print("Best KNN R2:", knn_grid.best_score_)
print("Best KNN Params:", knn_grid.best_params_)
```

```
C:\Users\Cifer\anaconda3\Lib\site-packages\sklearn\impute\_base.py:598:
UserWarning: Skipping features without any observed values:
['cholesterol_level']. At least one non-missing value is needed for imputation
with strategy='median'.
  warnings.warn(
C:\Users\Cifer\anaconda3\Lib\site-packages\sklearn\impute\_base.py:598:
UserWarning: Skipping features without any observed values:
['cholesterol_level']. At least one non-missing value is needed for imputation
with strategy='median'.
  warnings.warn(
```

```
Best KNN R2: 0.5069823127373232
Best KNN Params: {'algorithm': 'ball_tree', 'n_neighbors': 5, 'weights':
'distance'}
```

0.4.5 Gradient Boosting

[111]:
```
from sklearn.impute import SimpleImputer
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import GridSearchCV

# Impute missing values in X_train
imputer = SimpleImputer(strategy='median')
X_train_imputed = imputer.fit_transform(X_train)
X_test_imputed = imputer.transform(X_test)

# Define the model
gbr = GradientBoostingRegressor(random_state=42)

# Parameters to tune
params = {
    'n_estimators': [100, 200, 500],
    'learning_rate': [0.01, 0.1, 0.05],
    'max_depth': [3, 5, 7],
    'subsample': [0.8, 1.0]
}
```

```python
# Perform grid search with the imputed training data
gbr_grid = GridSearchCV(gbr, params, cv=5, scoring='r2')
gbr_grid.fit(X_train_imputed, y_train)

# Best parameters and performance
print("Best Gradient Boosting params:", gbr_grid.best_params_)
print("Best Gradient Boosting R²:", gbr_grid.best_score_)
```

```
C:\Users\Cifer\anaconda3\Lib\site-packages\sklearn\impute\_base.py:598:
UserWarning: Skipping features without any observed values:
['cholesterol_level']. At least one non-missing value is needed for imputation
with strategy='median'.
  warnings.warn(
C:\Users\Cifer\anaconda3\Lib\site-packages\sklearn\impute\_base.py:598:
UserWarning: Skipping features without any observed values:
['cholesterol_level']. At least one non-missing value is needed for imputation
with strategy='median'.
  warnings.warn(
```

```
Best Gradient Boosting params: {'learning_rate': 0.05, 'max_depth': 5,
'n_estimators': 100, 'subsample': 0.8}
Best Gradient Boosting R²: 0.9579335678321742
```

0.4.6 Best Model Fit After Model Tuning

- Gradient Boosting emerged as the best-performing model.
- Best R^2: The Gradient Boosting model achieved the highest R^2 score during tuning, indicating that it explains the largest proportion of variance in the target variable.
- Hyperparameters Tuned: n_estimators (number of trees), learning_rate (shrinkage rate), max_depth (depth of each tree), subsample (fraction of samples used).
- Random Forest: Despite being a strong competitor, it likely underperformed compared to Gradient Boosting due to limitations in handling complex patterns within the dataset.
- Decision Tree: While interpretable, it often suffers from overfitting or underfitting depending on its depth.
- KNN: Generally less effective with high-dimensional data, leading to lower R^2 scores.
- Linear Regression and Regularized Variants (Ridge/Lasso): Performed reasonably but lacked the flexibility to capture non-linear relationships.

0.4.7 Intepretation and Implication on Business

- Accurate Risk Prediction: With more accurate predictions of insurance costs based on customer profiles (health, lifestyle, etc.), the company can more effectively manage risk.
- Personalized Insurance Pricing: Gradient Boosting allows the business to implement highly personalized pricing based on numerous factors (e.g., weight, heart disease history, number

of checkups).

- Improved Customer Segmentation: The model can be used to segment customers based on risk factors (e.g., health, exercise habits, smoking status), leading to more effective marketing and sales strategies.
- Prevention Strategies and Incentives: The model highlights preventive behaviors like regular doctor visits and checkups that reduce insurance costs. These insights can be used to design wellness programs or incentives for customers.
- Resource Allocation: With better risk prediction, insurers can allocate resources more effectively. High-risk customers can receive more attention, while low-risk customers can be managed with fewer resources.
- Fraud Detection: Highly accurate models like Gradient Boosting can also help in detecting anomalous patterns that might suggest fraud (e.g., inconsistent reporting of health conditions).
- Data-Driven Decision Making: Using Gradient Boosting for predictive modeling equips the business with a reliable tool for decision-making across multiple areas like underwriting, claims management, and product development.

[ ]:

```
```